Circuit Design
Схемотехника

Melis Sydykbekovich Osmonov,
Candidate of Technical Sciences,
KRSU,IVT

© Osmonov M.S. KRSU

Combinational Logic
Комбинационная логика

© Osmonov M.S.

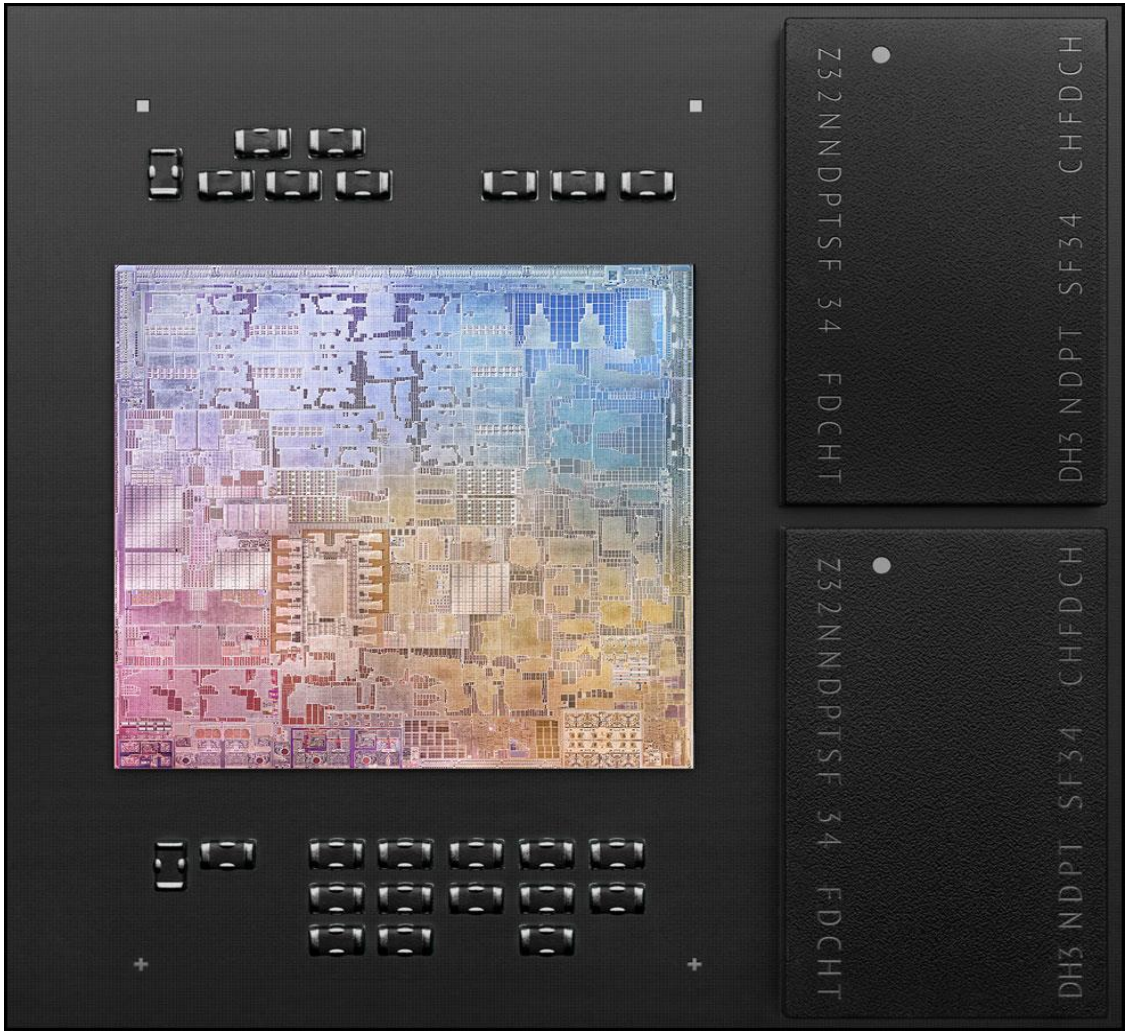
Modern General-Purpose Microprocessors

5-nanometer process

The first personal computer chip built with this cutting-edge technology.

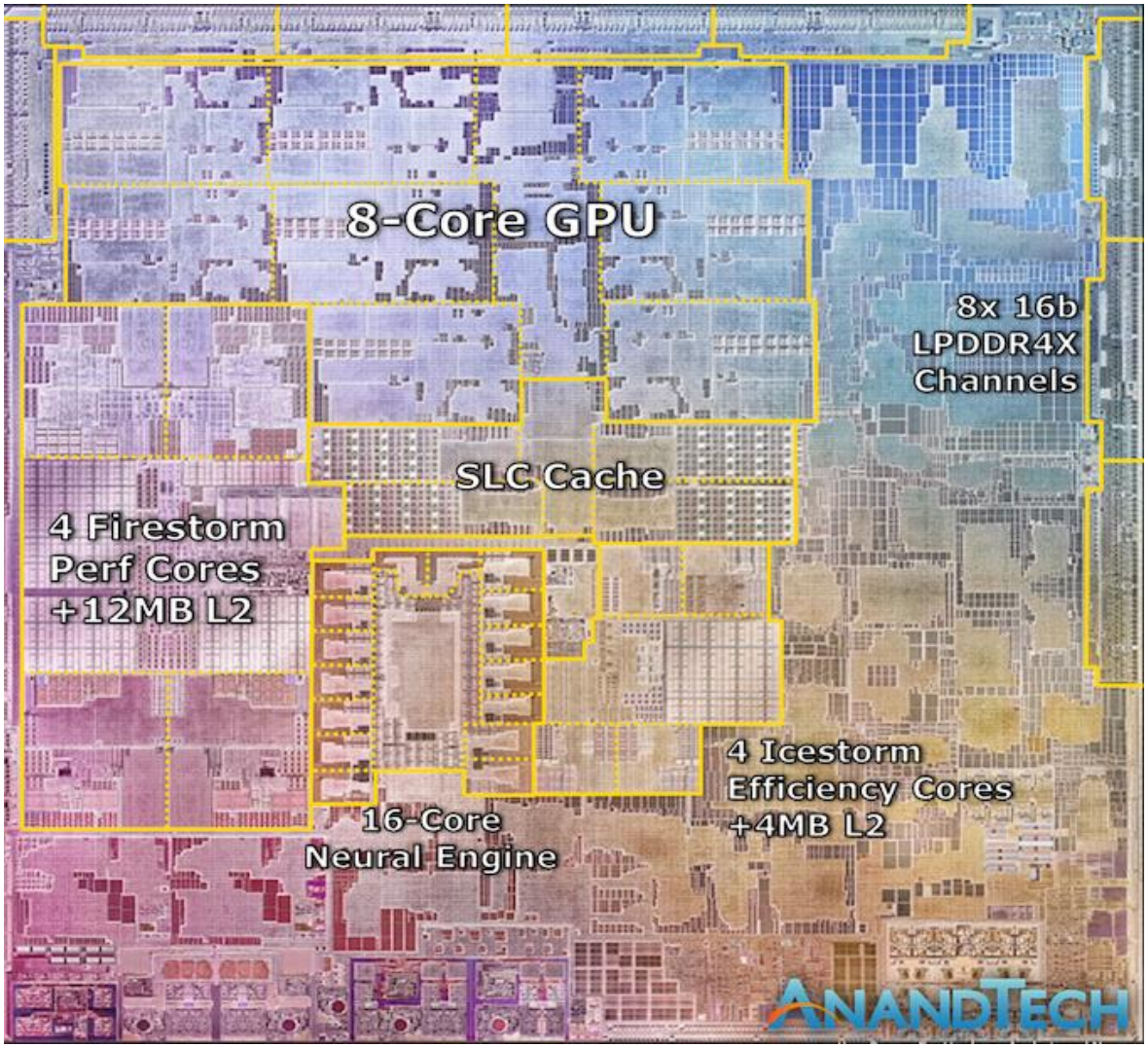
16 billion transistors

The most we've ever put into a single chip.



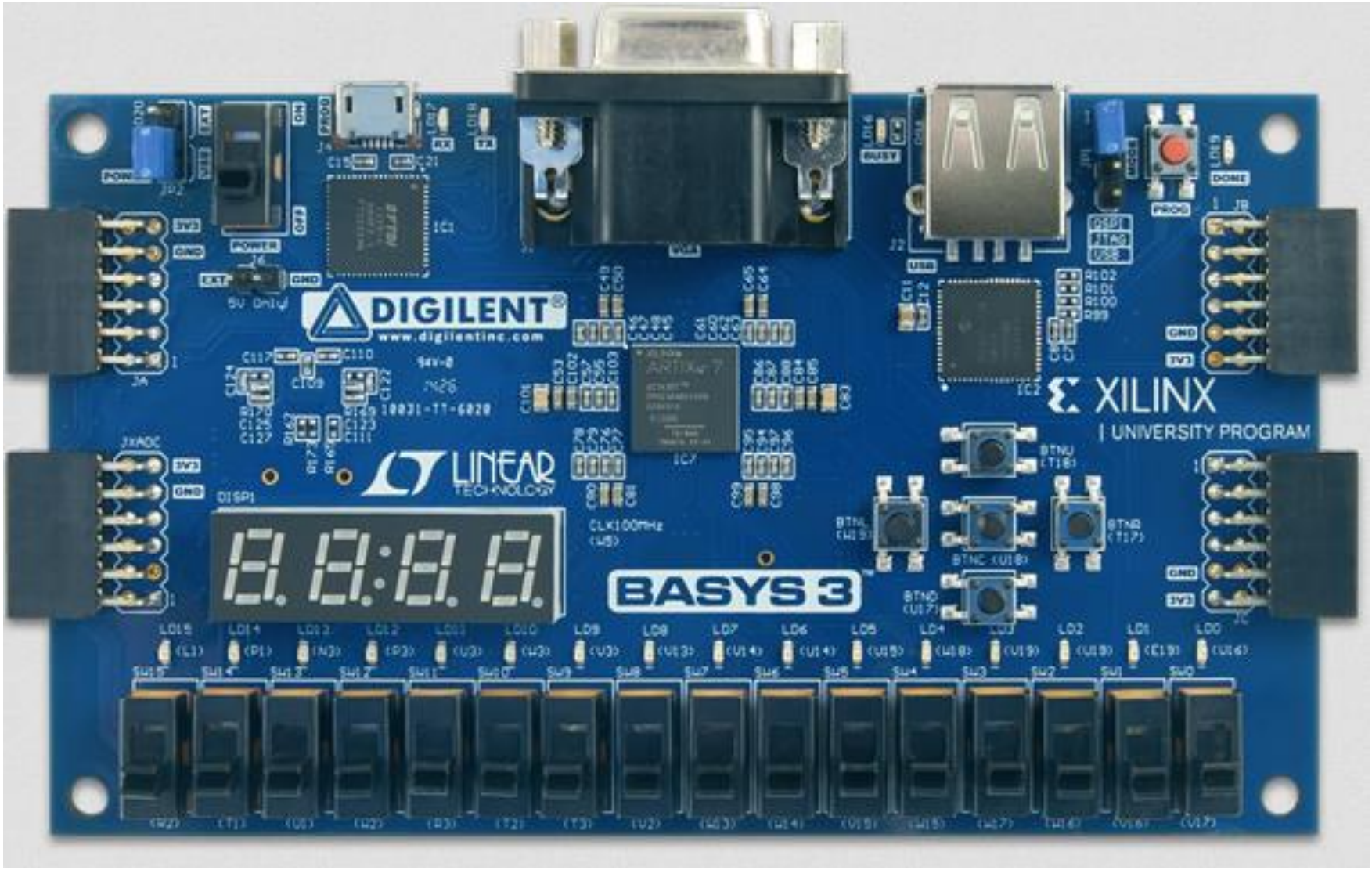
Source: https://www.apple.com/mac/m1/

Modern General-Purpose Microprocessors



Source: https://www.anandtech.com/show/16252/mac-mini-apple-m1-tested

FPGAs

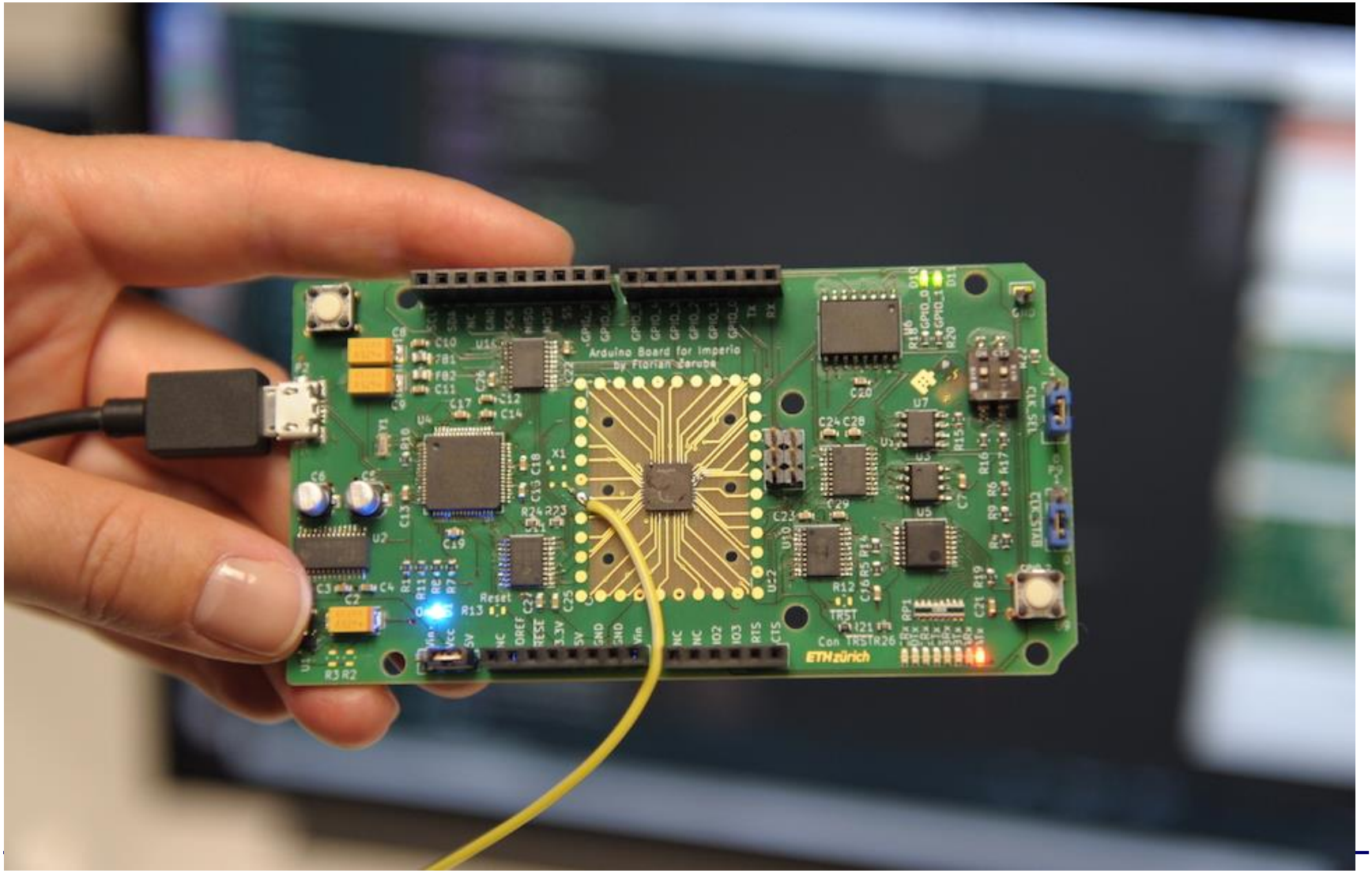

DIGILENT
www.digilentinc.com
XILINX
UNIVERSITY PROGRAM
LINEAR TECHNOLOGY
BASYS 3

Modern FPGAs



Source: https://www.mouser.ch/new/xilinx/xilinx-zynq-7000-zc702-eval-kit/

Special-Purpose ASICs (App-Specific Integrated Circuits)



Modern Special-Purpose ASICs

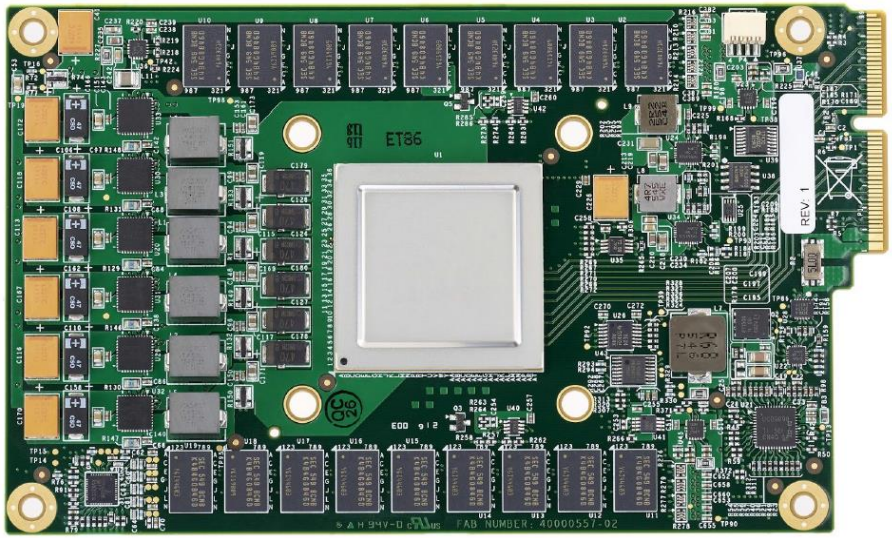


Figure 3. TPU Printed Circuit Board. It can be inserted in the slot for an SATA disk in a server, but the card uses PCIe Gen3 x16.

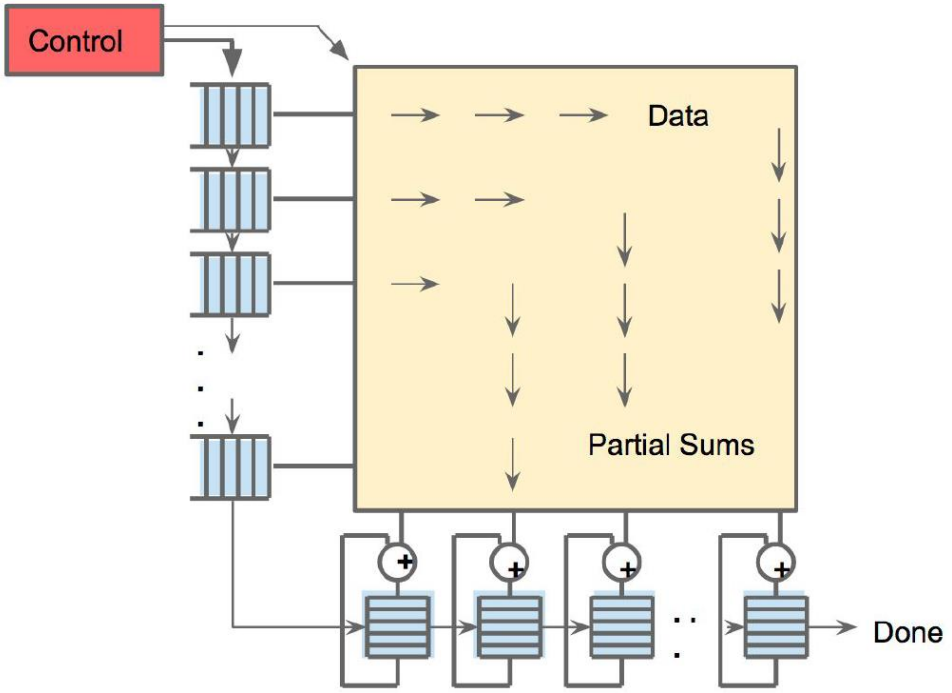
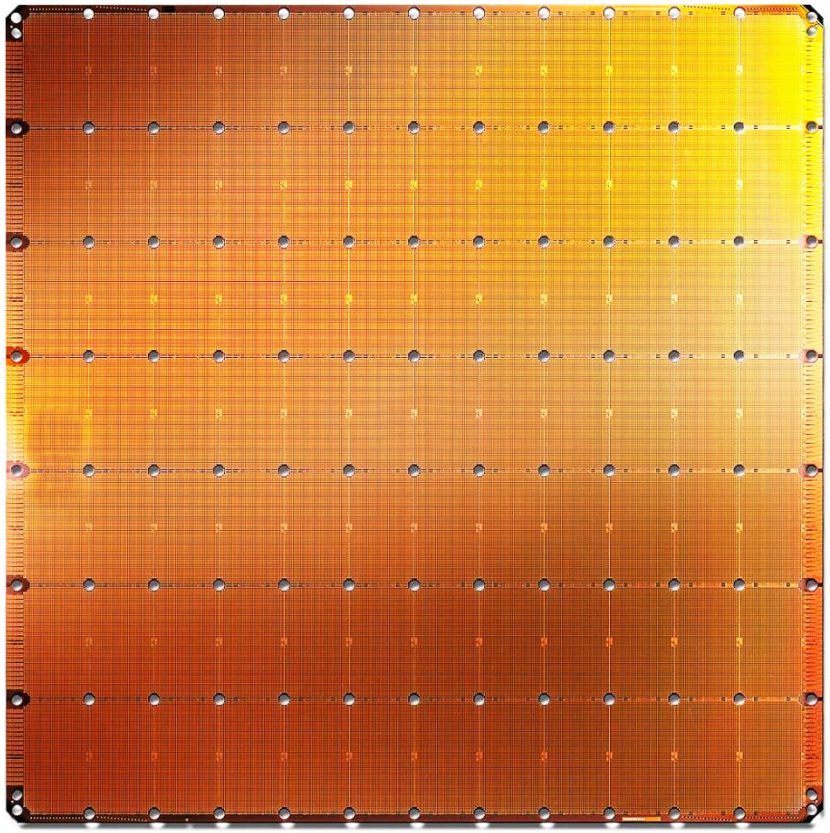


Figure 4. Systolic data flow of the Matrix Multiply Unit. Software has the illusion that each 256B input is read at once, and they instantly update one location of each of 256 accumulator RAMs.

Jouppi et al., "In-Datacenter Performance Analysis of a Tensor Processing Unit", ISCA 2017.

Modern Special-Purpose ASICs



Cerebras WSE
1.2 Trillion transistors
46,225 mm^2

- The largest ML accelerator chip
- 400,000 cores

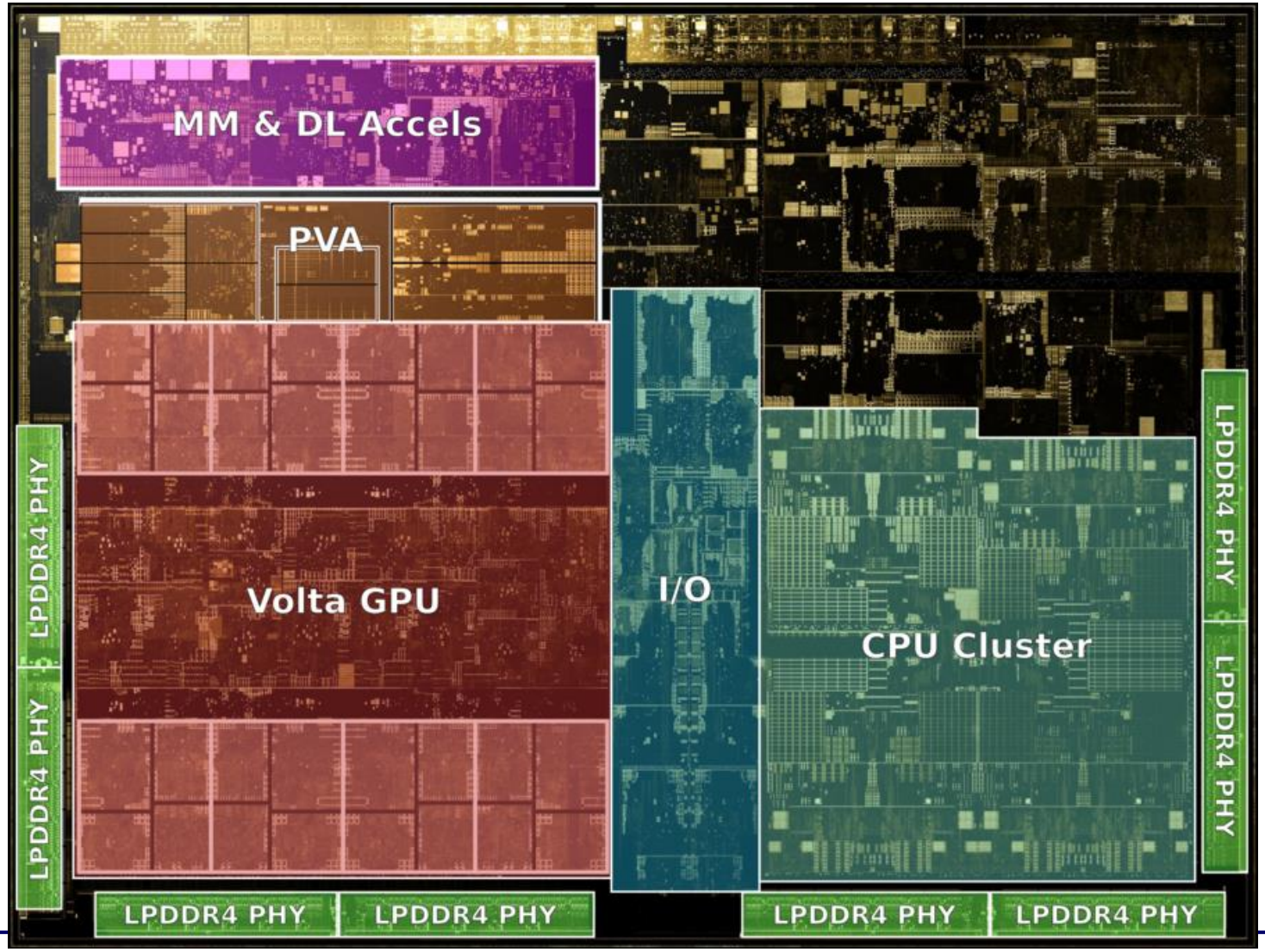


Largest GPU
21.1 Billion transistors
815 mm^2
NVIDIA TITAN V

https://www.anandtech.com/show/14758/hot-chips-31-live-blogs-cerebras-wafer-scale-deep-learning

https://www.cerebras.net/cerebras-wafer-scale-engine-why-we-need-big-chips-for-deep-learning/

Modern GPUs



Source: https://en.wikichip.org/wiki/nvidia/tegra/xavier

They All Look the Same

	Microprocessors	FPGAs	ASICs
			
In short:	Common building block of computers	Reconfigurable hardware, flexible	You customize everything

They All Look the Same

	Microprocessors	FPGAs	ASICs
			
In short:	Common building block of computers	Reconfigurable hardware, flexible	You customize everything
Program Development Time	minutes	days	months

They All Look the Same

	Microprocessors	FPGAs	ASICs
			
In short:	Common building block of computers	Reconfigurable hardware, flexible	You customize everything
Program Development Time	minutes	days	months
Performance	o	+	++

They All Look the Same

	Microprocessors	FPGAs	ASICs
			
In short:	Common building block of computers	Reconfigurable hardware, flexible	You customize everything
Program Development Time	minutes	days	months
Performance	o	+	++
Good for	Ubiquitous Simple to use	Prototyping Small volume	Mass production, Max performance
Programming	Executable file	Bit file	Design masks
Languages	C/C++/Java/…	Verilog/VHDL	Verilog/VHDL
Main Companies	Intel, ARM, AMD, Apple, NVIDIA	Xilinx, Altera	TSMC, Globalfoundries

They All Look the Same

Want to learn how these work

By programming these

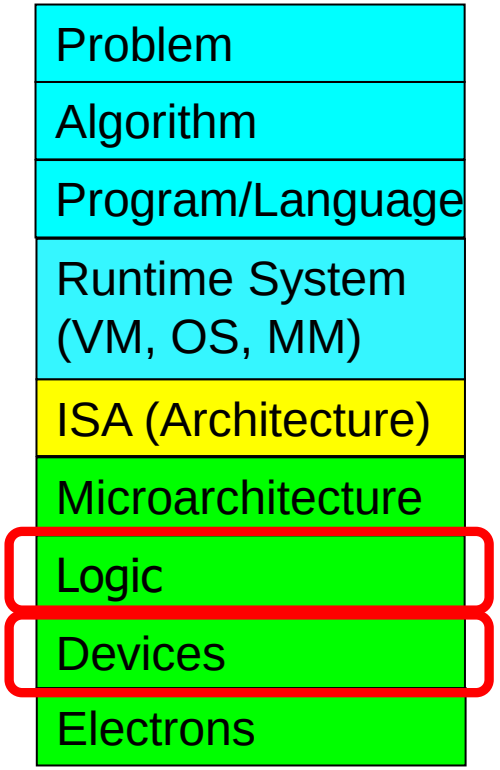
	Microprocessors	FPGAs	ASICs
			
In short:	Common building block of computers	Reconfigurable hardware, flexible	Customize everything
Program Development Time	minutes	days	months
Performance	o	+	++
Good for	Ubiquitous Simple to use	Prototyping	Mass production
Programming	Executable file	Using this language	
Languages	C/C++/Java/…	Verilog/VHDL	Verilog/VHDL
Main Companies	Intel, ARM, AMD, Apple, NVIDIA	Xilinx, Altera	TSMC, Globalfoundries

Building Blocks of Modern Computers

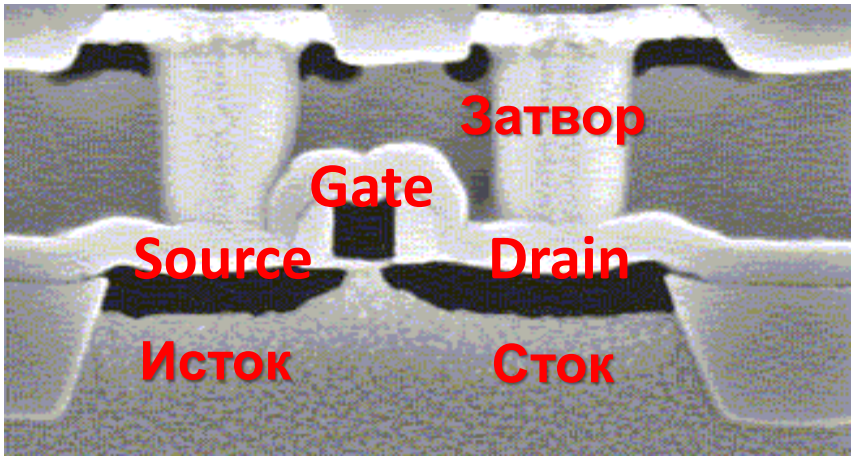
Transistors
Транзисторы

Транзисторы

- Компьютеры состоят из очень большого количества очень простых (и относительно простых) структур: транзисторов.
  - Микропроцессор Intel Pentium IV, впервые появившийся в продаже в 2000 году, состоял из более чем 42 миллионов МОП-транзисторов.
  - Apple M1 Max, выставленный на продажу в 2021 году, состоит из более чем 56 миллиардов МОП-транзисторов.
- В этой лекции
  - Как работает МОП-транзистор (как логический элемент)
  - Как эти транзисторы соединены в логические элементы
  - Как логические элементы соединяются между собой, образуя более крупные блоки, необходимые для построения компьютера.



МОП-транзистор



- Название образуется из:
  - Проводники (**М**еталл)
  - Изоляторы (**О**ксид)
  - **П**олупроводники
- Получаем транзистор (МОП)
- Почему это полезно?
  - Мы можем комбинировать многие из них, чтобы реализовать простые логические вентили.
- Электрические свойства полупроводников на основе оксидов металлов выходят далеко за рамки того, что мы изучаем в рамках этого курса.
  - Они ниже нашего самого низкого уровня абстракции.

Типы МОП-транзисторов

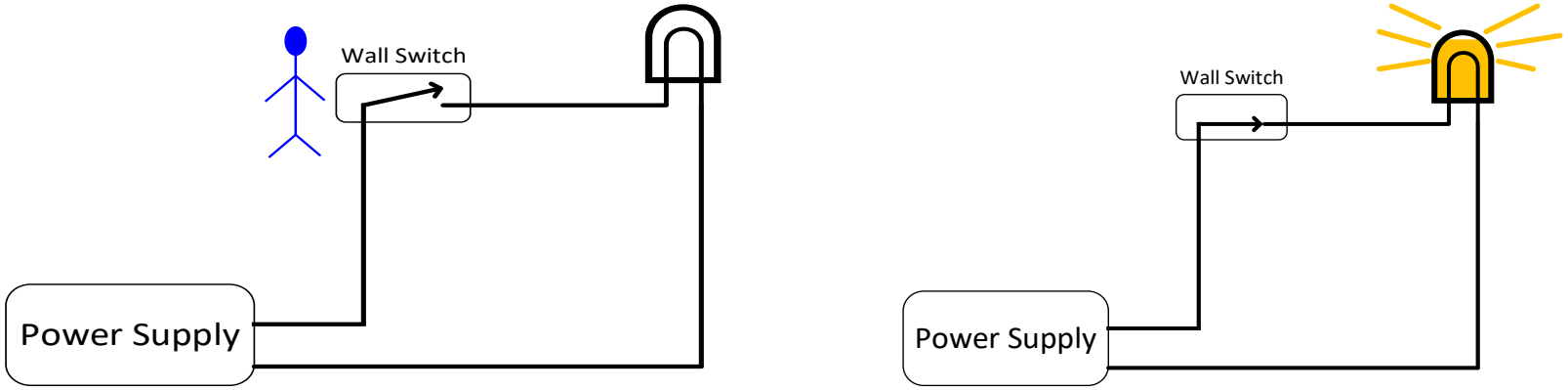
- Существует два типа МОП-транзисторов: ***n***-типа и ***p***-типа.



n-type p-type

- Их “логическая” работа очень похожа на работу настенных выключателей.

Как работает транзистор?



- Чтобы лампочка светилась, электроны должны течь по цепи.
- Чтобы электроны текли, должна быть замкнутая цепь от источника питания к лампе и обратно к источнику питания.
- Лампочку можно включать и выключать, просто манипулируя настенным выключателем, замыкая или размыкая цепь.

Как работает транзистор?

- Вместо настенного выключателя можно использовать МОП-транзистор *n-типа* или *p-типа*, чтобы замыкать или размыкать цепь.

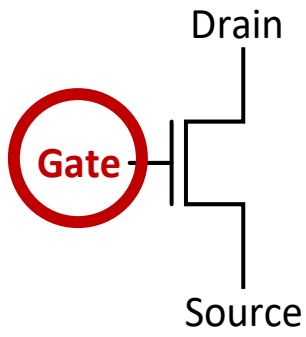


Схема МОП-транзистора n-типа

- Если на затвор (gate) транзистора *n-типа* подается высокое напряжение, соединение от истока (source) к стоку (drain) действует как кусок провода.

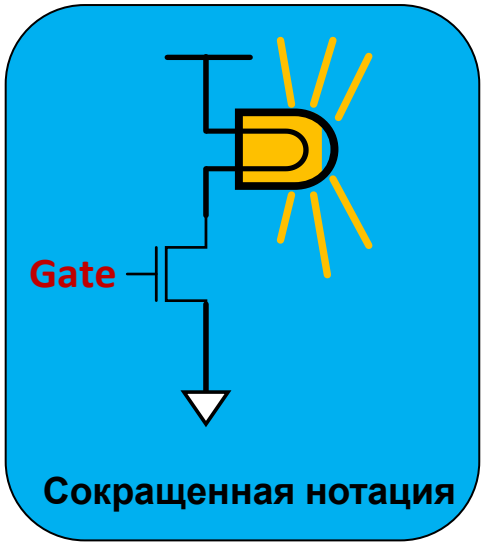
В зависимости от технологии, высокое напряжение может варьироваться от 0,3 В до 3 В

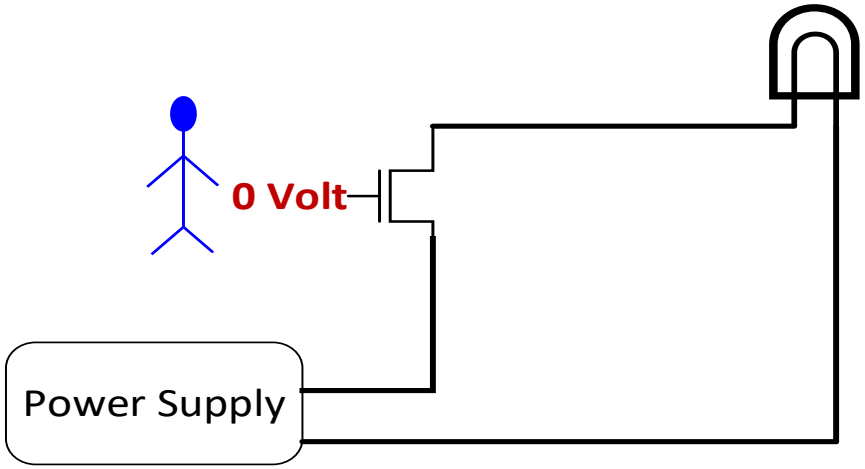
- Если на затвор (gate) транзистора *n-типа* подается нулевое напряжение, связь между истоком (source) и стоком (drain) разрывается.

© Osmonov M.S.
KRSU

Как работает транзистор?

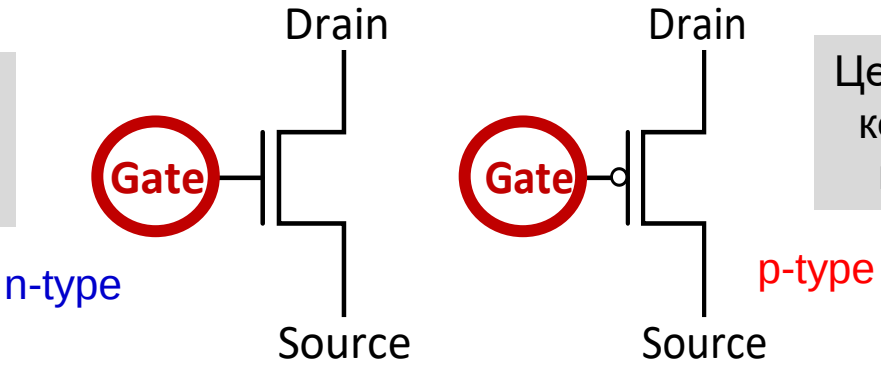
- Транзистор *n-типа* в схеме с батарейкой и лампочкой





- Транзистор *p-типа* работает по принципу, противоположному транзистору *n-типа*.

Цепь замыкается, когда на затвор подается напряжение 3 В.



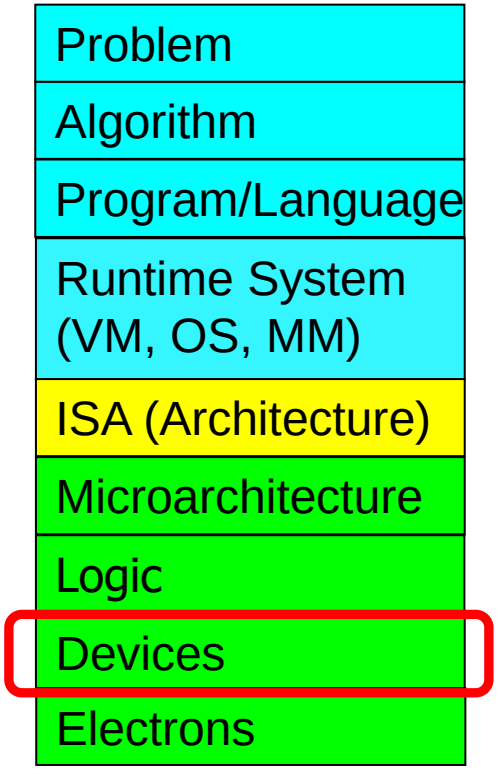
Цепь замыкается, когда на затвор подается 0 В.

Logic Gates
Логические вентили

На уровень выше в абстракции

- **Теперь вы знаете как работает МОП-транзистор.**
- Как построить логику из МОП-транзисторов?

- Рассмотрим как создаются базовые логические структуры из отдельных МОП-транзисторов.
- Эти логические единицы называются логическими вентилями (logic gates).
- Они реализуют простые булевы функции.



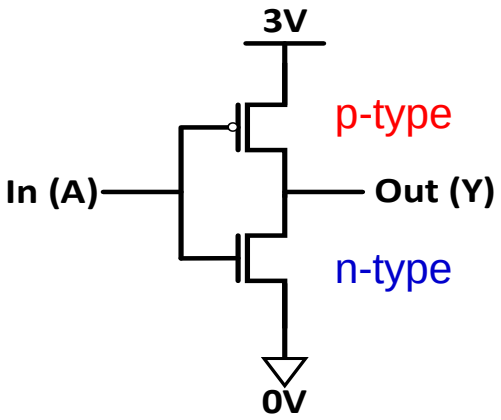
© Osmonov M.S.
KRSU

Создание логических блоков с использованием технологии КМОП (CMOS)

- В современных компьютерах используются транзисторы как n-типа, так и p-типа, т.е. технология комплементарных МОП (КМОП).

nМОП + pМОП = КМОП

- Простейшая логическая структура, используемая в современных компьютерах

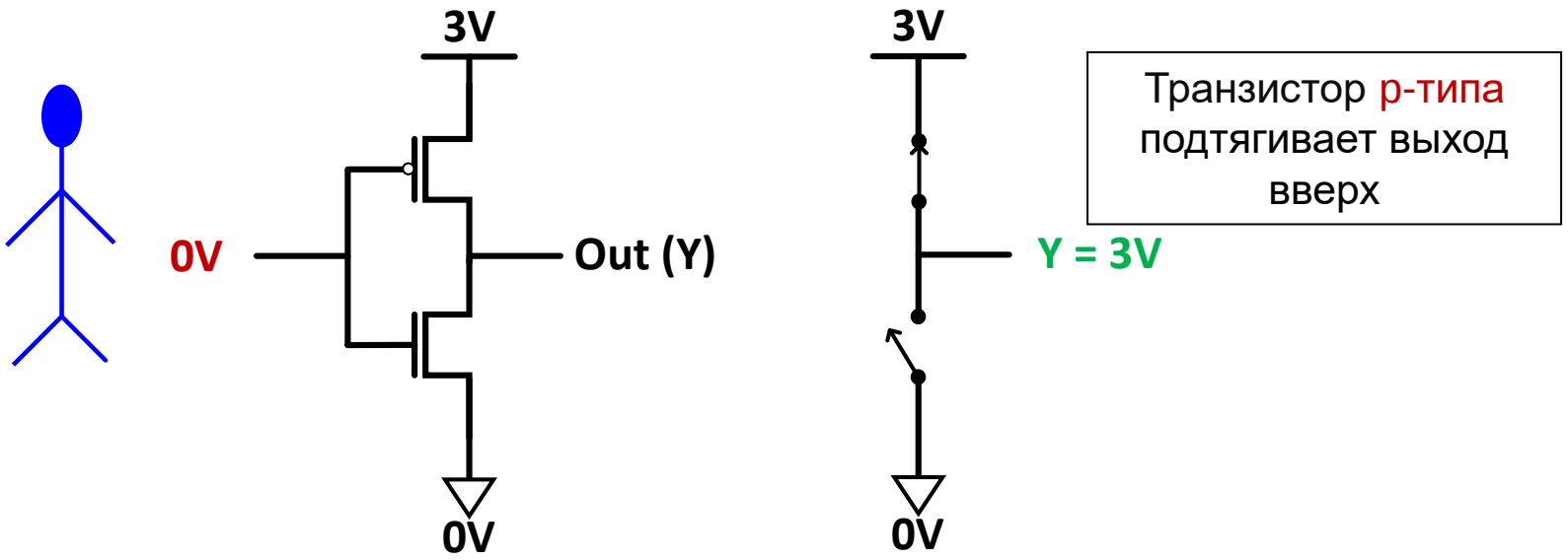


Как работает эта схема?

© Osmonov M.S.
KRSU

Функционирование КМОП-схемы

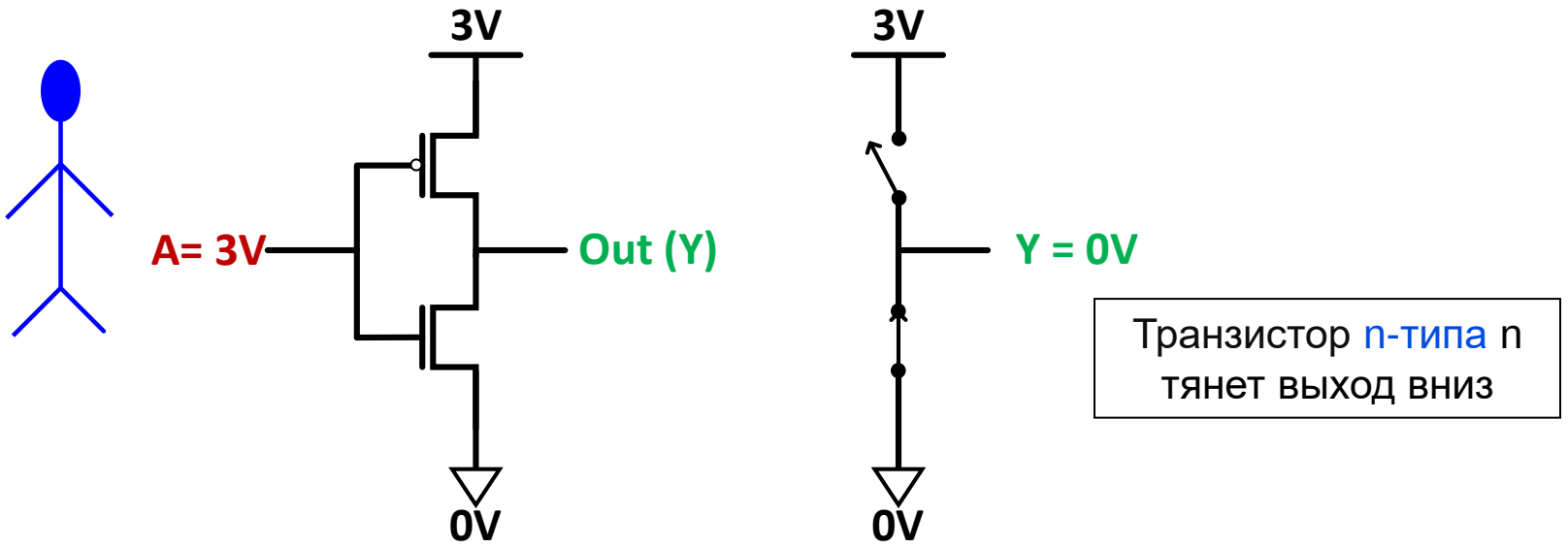
- Что происходит, когда вход подключен к 0 В?



p-type transistors are good at pulling u**p** the voltage

Функционирование КМОП-схемы

- Что происходит, когда вход подключен к 3 В?

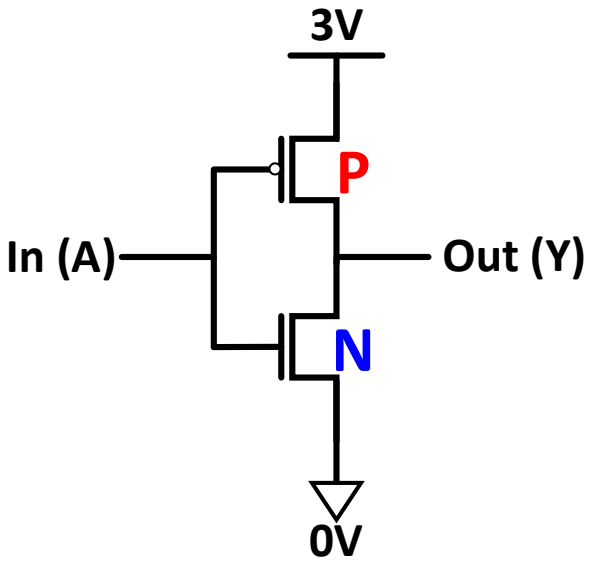


Транзистор n-типа n тянет выход вниз

n-type transistors are good at pulling down the voltage

КМОП вентиль **NOT**

- Это КМОП вентиль **NOT**.
- Почему он называется **NOT (НЕ) ?**
  - Если A = 0 В, то Y = 3 В.
  - Если A = 3 В, то Y = 0 В.
- Цифровая схема: одна из возможных интерпретаций
  - Интерпретировать 0V как значение логического (двоичного) 0
  - Интерпретировать 3V как значение логической (двоичной) 1



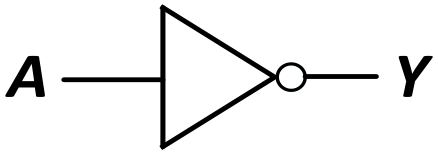
$$Y = \bar{A}$$

A	P	N	Y
0	**ON**	OFF	1
1	OFF	**ON**	0

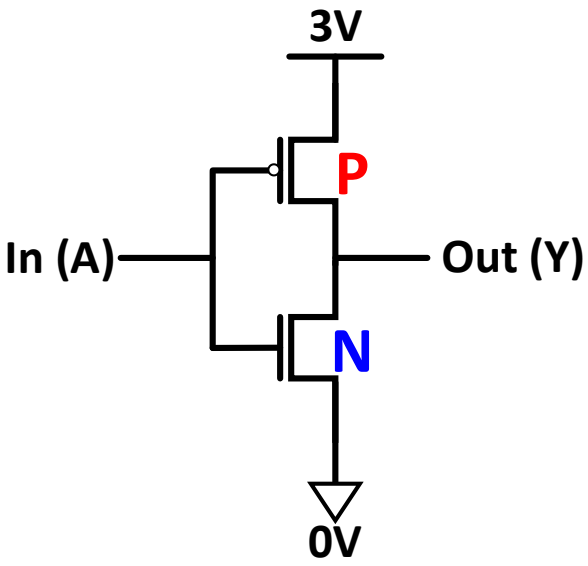
© Osmonov M.S.
KRSU

КМОП вентиль **NOT**

- Это КМОП вентиль **NOT**.
- Почему он называется **NOT (НЕ) ?**
  - Если A = 0 В, то Y = 3 В.
  - Если A = 3 В, то Y = 0 В.
- Цифровая схема: одна из возможных интерпретаций
  - Интерпретировать 0V как значение логического (двоичного) 0
  - Интерпретировать 3V как значение логической (двоичной) 1



Называется вентилем НЕ или инвертором



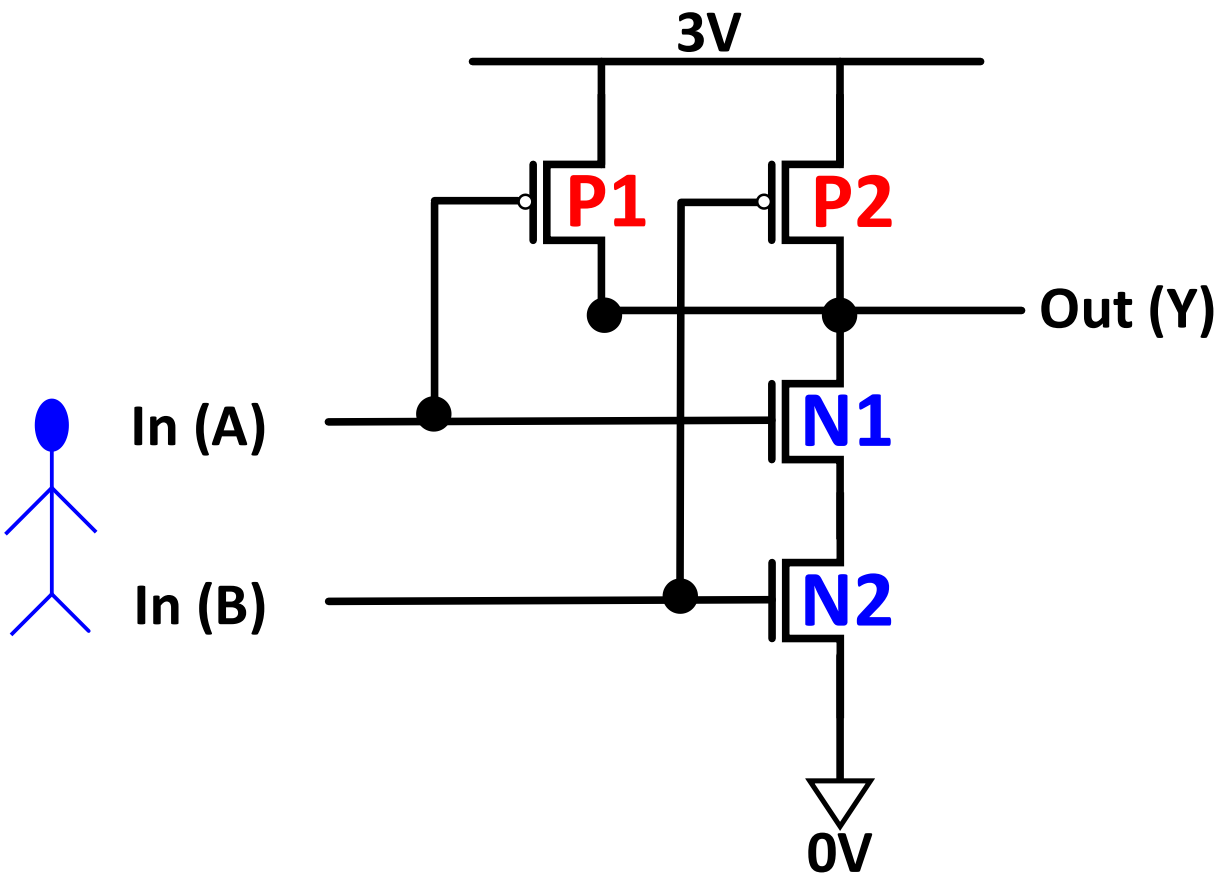
$$Y = \bar{A}$$

Таблица истинности: показывает, значение логического выхода схемы для каждого возможного значения логическо

A	Y
0	1
1	0

Еще один вариант КМОП вентиля

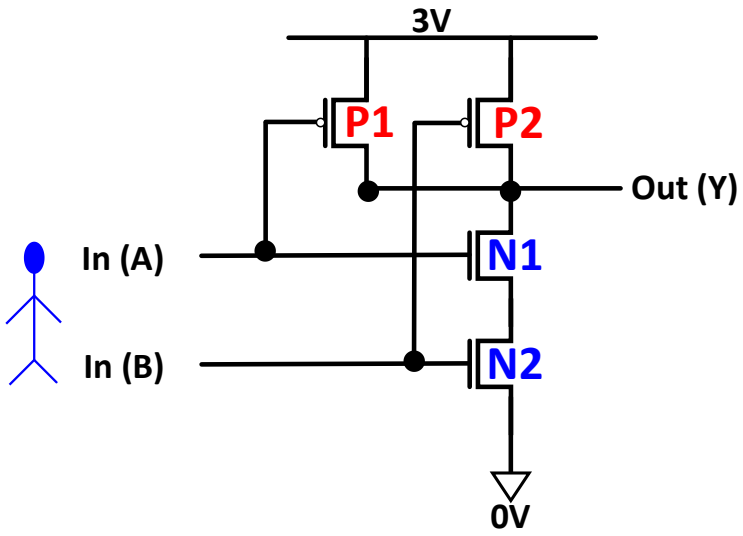
- Вариант более сложного вентиля!



© Osmonov M.S.
KRSU

КМОП вентиль **NAND (И-НЕ)**

- Вариант более сложного вентиля!



$$Y = \overline{A \cdot B} = \overline{AB}$$

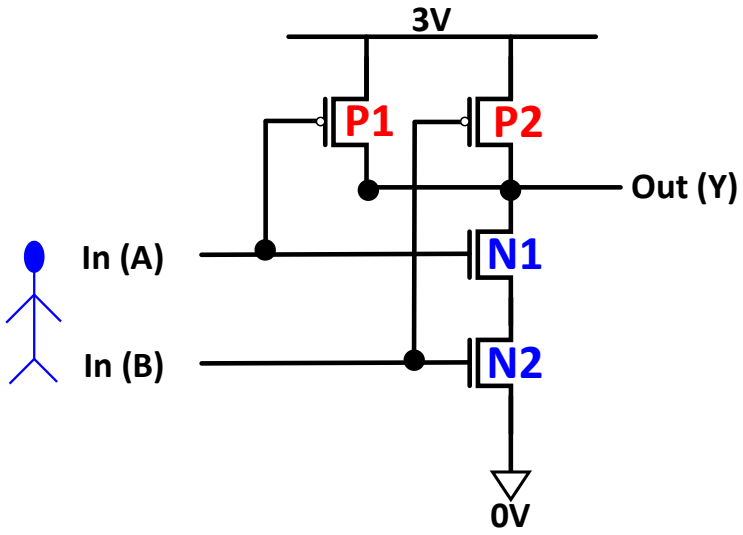
A	B	P1	P2	N1	N2	Y
0	0	**ON**	**ON**	OFF	OFF	1
0	1	**ON**	OFF	OFF	ON	1
1	0	OFF	**ON**	ON	OFF	1
1	1	OFF	OFF	**ON**	**ON**	0

- P1 и P2 соединены параллельно; **только один должен быть ON (включен), чтобы поднять Out (выход) до 3 В**
- N1 b N2 соединены последовательно; **оба должны быть ON (включены), чтобы Out (выход) сбросился до 0 В**

© Osmonov M.S.
KRSU

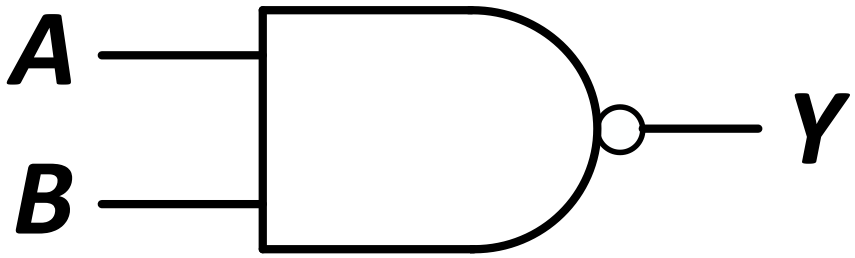
КМОП вентиль **NAND (И-НЕ)**

- Вариант более сложного вентиля!



$$Y = \overline{A \cdot B} = \overline{AB}$$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0



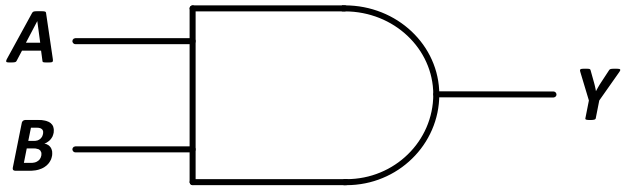
© Osmonov M.S.
KRSU

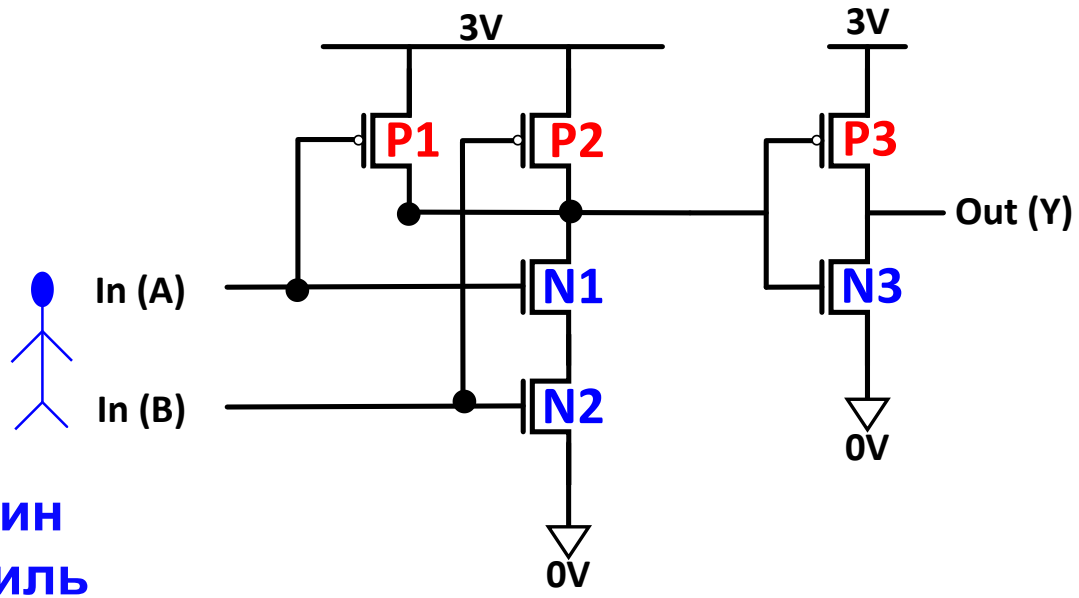
КМОП вентиль **AND (И)**

- Реализация вентиля **AND**

$$Y = \overline{A \cdot B} = \overline{AB}$$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

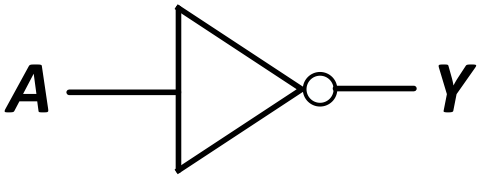


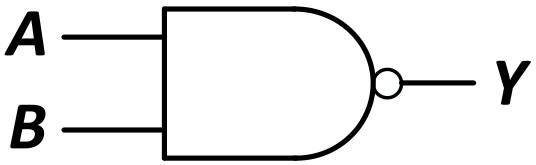


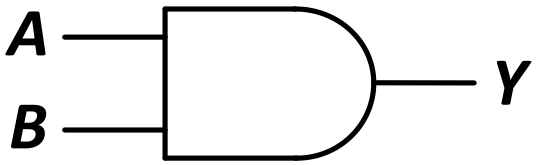
Вентиль AND можно реализовать, используя один вентиль NAND и один вентиль NOT

© Osmonov M.S.
KRSU

КМОП вентили **NOT**, **NAND**, **AND**



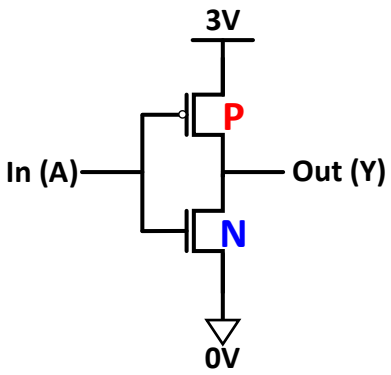


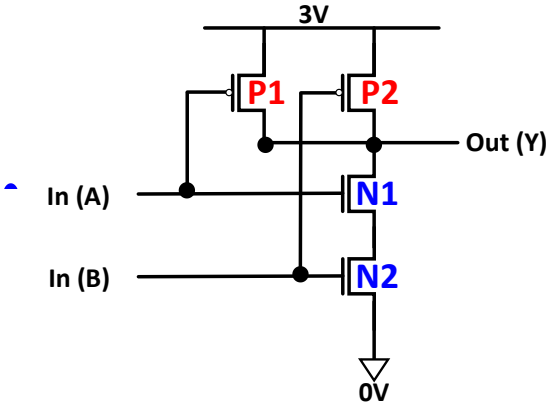


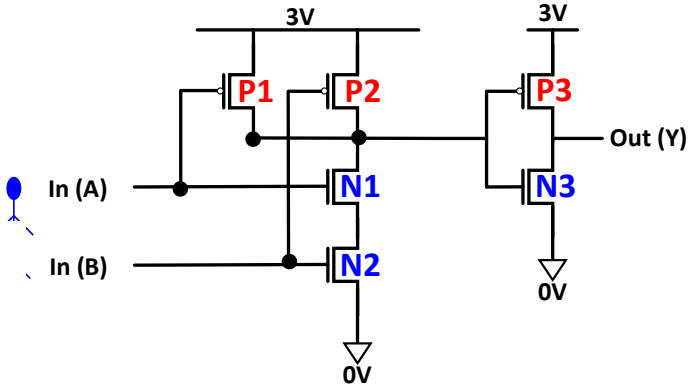
A	Y
0	1
1	0

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

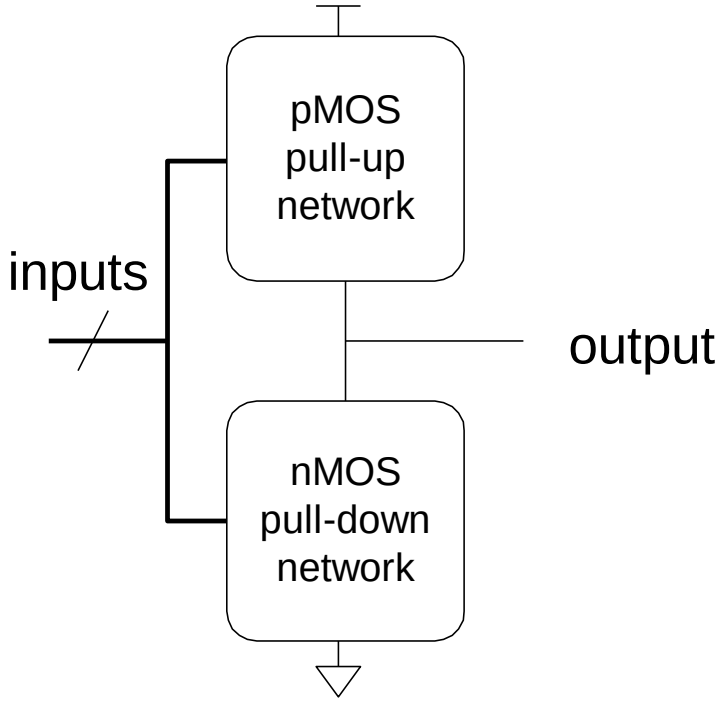






Обобщенная структура КМОП вентилей

- Общая форма, используемая для построения любого инвертирующего логического элемента, например: НЕ, НЕ-И или ИЛИ-НЕ.
  - Сети (networks) могут состоять из транзисторов, соединенных последовательно или параллельно.
  - Когда транзисторы соединены параллельно, сеть включена (**ON**), если один из транзисторов включен (**ON**).
  - Когда транзисторы соединены последовательно, сеть включена (**ON**), только тогда, когда включены (**ON**) все транзисторы.

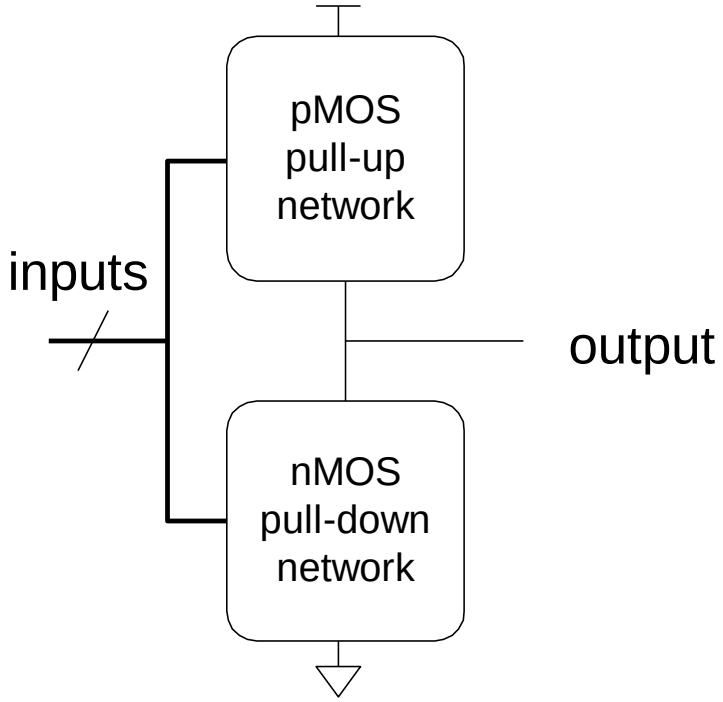


pMOS transistors are used for pull-up

nMOS transistors are used for pull-down

Обобщенная структура КМОП вентилей (2)

- Общая форма, используемая для построения любого инвертирующего логического элемента, например: НЕ, НЕ-И или ИЛИ-НЕ.
  - Сети (networks) могут состоять из транзисторов, соединенных последовательно или параллельно.
  - Когда транзисторы соединены параллельно, сеть включена (**ON**), если один из транзисторов включен (**ON**).
  - Когда транзисторы соединены последовательно, сеть включена (**ON**), только тогда, когда включены (**ON**) все транзисторы.

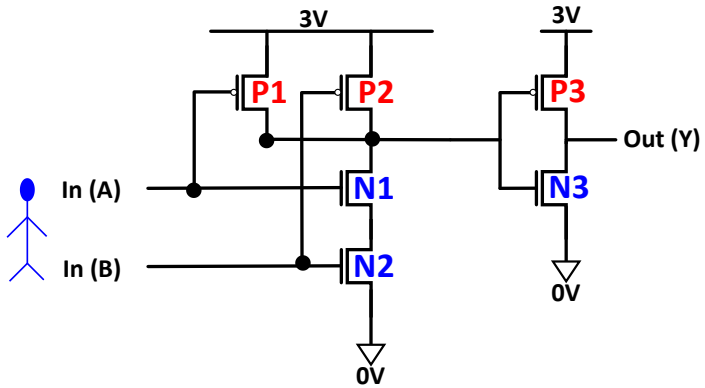


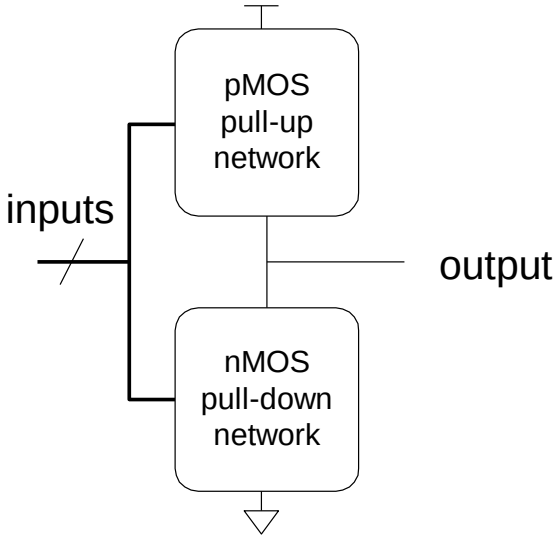
pMOS transistors are used for pull-up

nMOS transistors are used for pull-down

Почему именно такая структура?

- МОП-транзисторы не являются идеальными переключателями
  - Транзисторы p-МОП хорошо передают 1, но плохо передают 0
  - n-МОП-транзисторы хорошо передают 0, но плохо передают 1
  - Транзисторы p-МОП хорошо «подтягивают» выход
  - Транзисторы n-МОП хорошо «оттягивают» выход

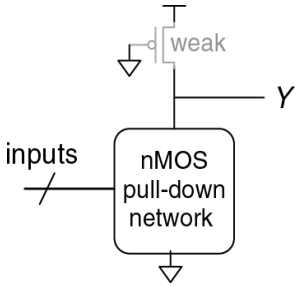




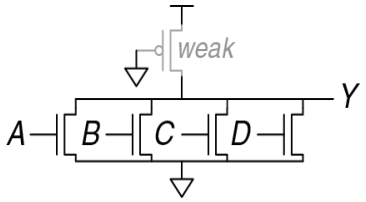
© Osmonov M.S.
KRSU

Задержки

- Какой из них быстрее?
  - Транзисторы, соединенные последовательно
  - Транзисторы, соединенные параллельно
- Последовательно включенные транзисторы передают сигнал медленнее, чем транзисторы, включенные параллельно
  - Аналогично тому, как сопротивление резисторов, включенных последовательно, будет больше, чем сопротивление резисторов, включенных параллельно.
- Как уменьшить эту задержку?
  - Пример см. в разделе 1.7.8 H&H: логика псевдо-nMOS.



Обобщенный псевдо n-МОП-вентиль



Псевдо n-МОП-вентиль ИЛИ-НЕ с четырьмя входами

Потребляемая мощность

- Dynamic Power Consumption
  - $C * V^2 * f$
    - C = capacitance of the circuit (wires and gates)
    - V = supply voltage
    - f = charging frequency of the capacitor
- Static Power consumption
  - $V * I_{leakage}$
    - supply voltage * leakage current
- Energy Consumption
  - Power * Time
- See more in H&H Chapter 1.8

Потребляемая мощность

- *Потребляемая* мощность – это количество энергии, потребляемой системой в единицу времени.
- Энергопотребление имеет большое значение в цифровых системах.
  - Время автономной работы без подзарядки батареи любого портативного устройства
  - Стоимость электроэнергии и т.д.
- Цифровая система потребляет энергию как в динамическом режиме, когда выполняет какие-либо операции, так и в статическом, когда система находится в состоянии покоя *(idle)*.
- Мощность потребляемой энергии в динамическом режиме:

$$P_{dynamic} = C \cdot V^2 \cdot f ,$$

где C – емкость цепи (провода и вентили), V – напряжение питания, а f – частота зарядки конденсаторов

- Мощность потребляемой энергии в статическом режиме:

$P_{static} = V \cdot I_{leakage}$, где $I_{leakage}$ - ток утечки (ток покоя)

Общие логические элементы (вентили)

Buffer



A	Z
0	0
1	1

AND



A	B	Z
0	0	0
0	1	0
1	0	0
1	1	1

OR



A	B	Z
0	0	0
0	1	1
1	0	1
1	1	1

XOR



A	B	Z
0	0	0
0	1	1
1	0	1
1	1	0

Inverter



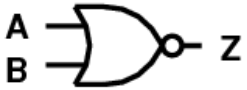
A	Z
0	1
1	0

NAND



A	B	Z
0	0	1
0	1	1
1	0	1
1	1	0

NOR



A	B	Z
0	0	1
0	1	0
1	0	0
1	1	0

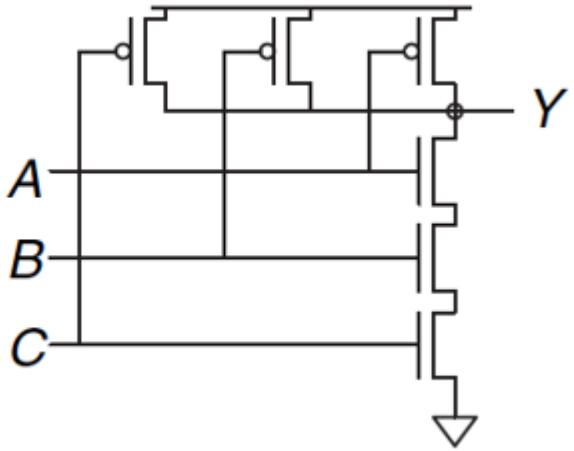
XNOR



A	B	Z
0	0	1
0	1	0
1	0	0
1	1	1

Большие вентили

- Вентили могут быть более чем на 2 входа
  - Например, 3-х входовый вентиль **AND** или 10-ти входовый вентиль **NOR**
- Схема 3-х входового вентиля NAND показана ниже:

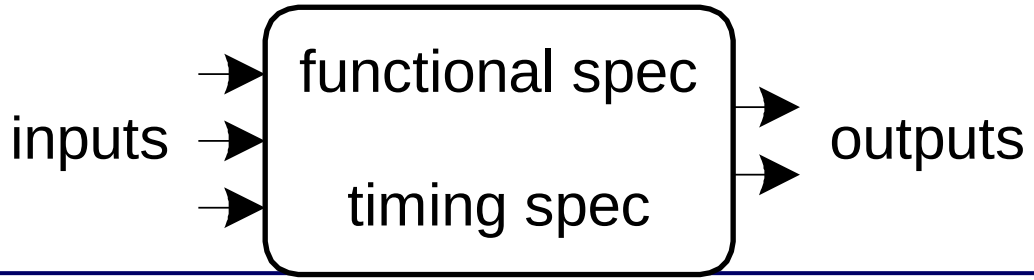


Combinational Logic Circuits
Комбинационные логические схемы

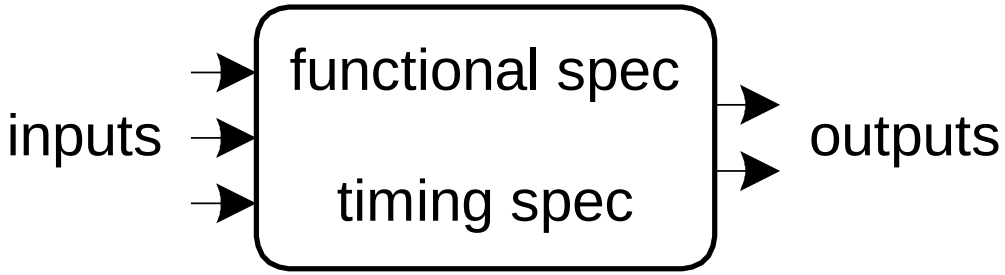
© Osmonov M.S.
IUCA

Логические схемы

- В цифровой электронике под *схемой* понимают электрическую цепь, которая обрабатывает дискретные сигналы.
- Такую схему можно рассматривать как «черный ящик», как показано на рисунке ниже, при этом схема имеет:
  - Один или более дискретных *входов*;
  - Один или более дискретных *выходов*;
  - *Функциональную спецификацию* (*functional specification*), описывающую взаимосвязь между входами и выходами;
  - *Временну́ю спецификацию* (*timing specification*), описывающую задержку между изменением сигналов на входе и откликом выходного сигнала.



Типы логических схем



- Комбинационная логика
  - Без памяти
  - Выходы зависят только от текущих значений на входах; другими словами, такие схемы комбинируют текущие значения входных сигналов для вычисления значения на выходе.
  - В некоторых книгах под названием "Комбинаторная логика«
- Последовательная логика
  - Имеет память
  - Структура хранит историю
    - Может «хранить» значения данных
  - Выходы определяются предыдущими (историческими) и текущими значениями входов

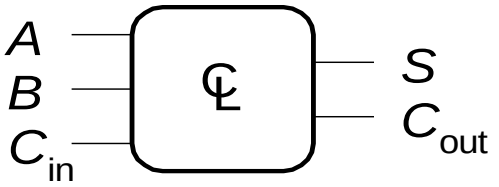
Boolean Logic Equations
Булевы логические уравнения

© Osmonov M.S.
IUCA

Функциональная спецификация

- Функциональная спецификация комбинационной схемы описывает зависимость значений на выходах от текущих входных значений
- Что мы подразумевается под словом “функцией”?
  - Уникальное сопоставление входных значений с выходными значениями
  - Одни и те же входные значения всегда дают одно и то же выходное значение.
  - Без памяти (не зависит от истории ввода значений)
- Пример (однобитный сумматор)

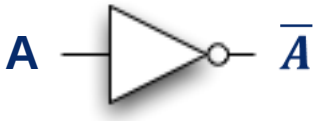
$$S = F(A, B, C_{in})$$
$$C_{out} = G(A, B, C_{in})$$



$$S = A \oplus B \oplus C_{in}$$
$$C_{out} = AB + AC_{in} + BC_{in}$$

Простые уравнения: NOT / AND / OR

$\overline{A}$ *(читается “не A”)* - равен 1, если A равен 0



A	$\overline{A}$
0	1
1	0

A • B *(читается “A и B”)* - равен 1, если A и B оба равны 1



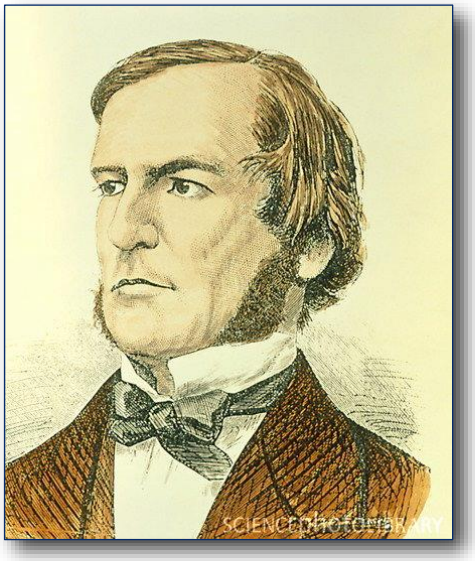
A	B	$A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

A + B *(читается “A или B”) - равен* 1, если A или (и) B равны 1



A	B	$A + B$
0	0	0
0	1	1
1	0	1
1	1	1

Булева алгебра



- Алгебра единиц и нулей
  - с операциями И, ИЛИ, НЕ
- С чего начать изучение
  - Аксиомы: основные вещи об объектах и операциях, которые просто считаются истинными сначала.
- Далее …
  - Законы и теоремы:
    - позволяют манипулировать булевыми выражениями
    - …также позволяет упрощать логические выражения
- Позже …
  - Более «сложные» свойства, полезные для управления цифровыми проектами, представленными в виде булевых уравнений.

Булева Алгебра: аксиомы

Формальная версия	*English version*
1. *B* contains at least two elements, *0* and *1*, such that $0 \neq 1$	Math formality...
2. *Closure* $a,b \in B$, (i) $a + b \in B$ (ii) $a \bullet b \in B$	Result of AND, OR stays in set you start with
3. *Commutative Laws*: $a,b \in B$, (i) $a + b = b + a$ (ii) $a \bullet b = b \bullet a$	For primitive AND, OR of 2 inputs, order doesn't matter
4. *Identities*: $0, 1 \in B$ (i) $a + 0 = a$ (ii) $a \bullet 1 = a$	There are identity elements for AND, OR, that give you back what you started with
5. *Distributive Laws*: (i) $a + (b \bullet c) = (a + b) \bullet (a + c)$ (ii) $a \bullet (b + c) = a \bullet b + a \bullet c$	• distributes over +, just like algebra …but + distributes over •, also (!!)
6. *Complement*: (i) $a + \bar{a} = 1$ (ii) $a \bullet \bar{a} = 0$	There is a complement element; AND/ORing with it gives the identity elm.

©

Boolean Algebra: Duality

- Observation
  - All the axioms come in "dual" form
  - Anything true for an expression also true for its dual
  - So any derivation you could make that is true, can be flipped into dual form, and it stays true
- Duality — More formally
  - A dual of a Boolean expression is derived by replacing
    - Every AND operation with... an OR operation
    - Every OR operation with... an AND
    - Every constant 1 with... a constant 0
    - Every constant 0 with... a constant 1
    - But don't change any of the literals or play with the complements!

Example

$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

$$\rightarrow\ a + (b \cdot c) = (a + b) \cdot (a + c)$$

Boolean Algebra: Useful Laws

	Dual	
Operations with 0 and 1:		
1. $X + 0 = X$	1D. $X \cdot 1 = X$	AND, OR with identities gives you back the original variable or the identity
2. $X + 1 = 1$	2D. $X \cdot 0 = 0$	
Idempotent Law:		
3. $X + X = X$	3D. $X \cdot X = X$	AND, OR with self = self
Involution Law:		
4. $\overline{(\overline{X})} = X$		double complement = no complement
Laws of Complementarity:		
5. $X + \overline{X} = 1$	5D. $X \cdot \overline{X} = 0$	AND, OR with complement gives you an identity
Commutative Law:		
6. $X + Y = Y + X$	6D. $X \cdot Y = Y \cdot X$	Just an axiom…

Useful Laws (continued)

Associative Laws:

7. $(X + Y) + Z = X + (Y + Z) = X + Y + Z$

7D. $(X \bullet Y) \bullet Z = X \bullet (Y \bullet Z) = X \bullet Y \bullet Z$

Parenthesis order does not matter

Distributive Laws:

8. $X \bullet (Y + Z) = (X \bullet Y) + (X \bullet Z)$

8D. $X + (Y \bullet Z) = (X + Y) \bullet (X + Z)$ Axiom

Simplification Theorems:

9. $X \bullet Y + X \bullet \overline{Y} = X$

9D. $(X + Y) \bullet (X + \overline{Y}) = X$

10. $X + X \bullet Y = X$

10D. $X \bullet (X + Y) = X$

11. $(X + \overline{Y}) \bullet Y = X \bullet Y$

11D. $(X \bullet \overline{Y}) + Y = X + Y$

Useful for simplifying expressions

Actually worth remembering — they show up a lot in real designs…

Boolean Algebra: Proving Things

- Proving theorems via axioms of Boolean Algebra

EX: Prove the theorem: $X \cdot Y + X \cdot \quad = X$

$$X \cdot \left(Y + \overline{Y}\right) = X \qquad \textbf{Distributive (5)}$$

$$X \cdot 1 = X \qquad \textbf{Complement (6)}$$

$$X = X \qquad \textbf{Identity (4)}$$

EX2: Prove the theorem: $X + X \cdot Y = X$

$$X \cdot 1 + X \cdot Y = X \qquad \textbf{Identity (4)}$$

$$X \cdot (1 + Y) = X \qquad \textbf{Distributive (5)}$$

$$X \cdot 1 = X \qquad \textbf{Identity (2)}$$

$$X = X \qquad \textbf{Identity (4)}$$

© Osmonov M.S.
KRSU

DeMorgan's Law: Enabling Transformations

DeMorgan's Law:

12. $\overline{(X + Y + Z + \cdots)} = \bar{X}.\bar{Y}.\bar{Z}. \ldots$

12D. $\overline{(X . Y . Z . \ldots)} = \bar{X} + \bar{Y} + \bar{Z} + \ldots$

- **Think of this as a transformation**
  - Let's say we have:

$$F = A + B + C$$

  - Applying DeMorgan's Law (12), gives us

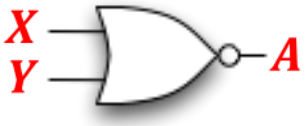
$$F = \overline{\overline{(A + B + C)}} = \overline{(\bar{A}.\bar{B}.\bar{C})}$$

At least one of A, B, C is TRUE --> It is **not** the case that A, B, C are **all** false

DeMorgan's Law (Continued)

These are conversions between different types of logic functions
They can prove useful if you do not have every type of gate…
Or, if some types of gates are more desirable to use than others…

$$A = \overline{(X + Y)} = \overline{X}\,\overline{Y}$$



NOR is equivalent to AND with inputs complemented

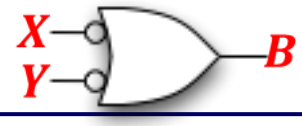


X	Y	$\overline{X+Y}$	$\overline{X}$	$\overline{Y}$	$\overline{X}\,\overline{Y}$
0	0	1	1	1	1
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	0

$$B = \overline{(XY)} = \overline{X} + \overline{Y}$$



NAND is equivalent to OR with inputs complemented



X	Y	$\overline{XY}$	$\overline{X}$	$\overline{Y}$	$\overline{X} + \overline{Y}$
0	0	1	1	1	1
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	0	0	0

Using Boolean Equations to Represent a Logic Circuit

Sum of Products Form: Key Idea

- Assume **we have the truth table of Boolean Function F**

- How do we express the function in terms of the inputs in a **standard** manner?

- Idea: **Sum of Products** form
- Express the truth table as a two-level Boolean expression
  - that contains **all** input variable combinations that result in a 1 output
  - If ANY of the combinations of input variables that results in a 1 is TRUE, then the output is 1
  - F = OR of all input variable combinations that result in a 1

Some Definitions (for a 3-Input Function)

- **Complement:** variable with a bar over it
  $\overline{A}, \overline{B}, \overline{C}$

- **Literal:** variable or its complement
  $A, \overline{A}, B, \overline{B}, C, \overline{C}$

- **Implicant:** product (AND) of literals
  $(A \cdot B \cdot \overline{C})$, $(\overline{A} \cdot C)$, $(B \cdot \overline{C})$

- **Minterm:** product (AND) that includes **all** input variables
  $(A \cdot B \cdot \overline{C})$, $(\overline{A} \cdot \overline{B} \cdot C)$, $(\overline{A} \cdot B \cdot \overline{C})$

- **Maxterm:** sum (OR) that includes **all** input variables
  $(A + \overline{B} + \overline{C})$, $(\overline{A} + B + \overline{C})$, $(A + B + \overline{C})$

Two-Level Canonical (Standard) Forms

- Truth table is the unique signature of a Boolean *function* …
  - But, it is an expensive representation

- A Boolean function can have many alternative Boolean expressions
  - i.e., many alternative Boolean expressions (and gate realizations) may have the same truth table (and function)
  - If they all say the same thing, why do we care?
    - Different Boolean expressions lead to different logic gate implementations → Different cost, latency, energy properties

- Canonical form: standard form for a Boolean expression
  - Provides a unique algebraic signature

Two-Level Canonical Forms: SOP

Sum of Products Form (SOP)

Also known as **disjunctive normal form** or **minterm expansion**

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$$F = \underset{011}{\bar{A}BC} + \underset{100}{A\bar{B}\bar{C}} + \underset{101}{A\bar{B}C} + \underset{110}{AB\bar{C}} + \underset{111}{ABC}$$

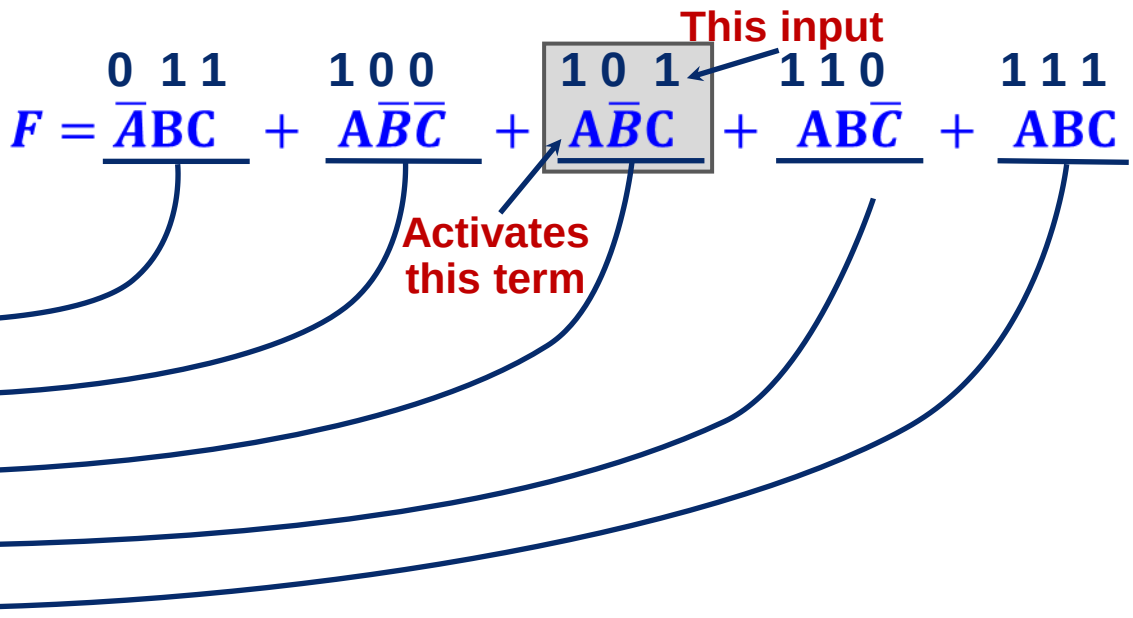
- Each row in a truth table has a minterm
- A minterm is a product (AND) of literals
- Each minterm is TRUE for that row (and only that row)

All Boolean equations can be written in SOP form

Find all the input combinations (minterms) for which the output of the function is TRUE.

SOP Form — Why Does It Work?

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1



- Only the shaded product term — $A\bar{B}C = 1 \cdot \bar{0} \cdot 1$ — will be 1
- No other product terms will "turn on" — they will all be 0
- So if inputs A B C correspond to a product term in expression,
  - We get 0 + 0 + … + 1 + … + 0 + 0 = 1 for output
- If inputs A B C do not correspond to any product term in expression
  - We get 0 + 0 + … + 0 = 0 for output

The function evaluates to TRUE (i.e., output is 1) if any of the Products (minterms) causes the output to be 1

Aside: Notation for SOP

- Standard "shorthand" notation
  - If we agree on the order of the variables in the rows of truth table…
    - then we can enumerate each row with the decimal number that corresponds to the binary number created by the input pattern

A	B	C	F	
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	
1	0	0	1	**100 = decimal 4 so this is minterm #4, or m4**
1	0	1	1	
1	1	0	1	
1	1	1	1	**111 = decimal 7 so this is minterm #7, or m7**

f = m3 + m4 + m5 + m6 + m7

= ∑m(3,4,5,6,7)

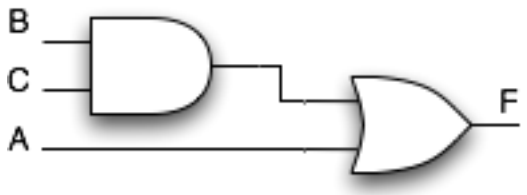
We can write this as a sum of products

Or, we can use a summation notation

Canonical SOP Forms

A	B	C	minterms
0	0	0	$\bar{A}\bar{B}\bar{C}$ = m0
0	0	1	$\bar{A}\bar{B}C$ = m1
0	1	0	$\bar{A}B\bar{C}$ = m2
0	1	1	$\bar{A}BC$ = m3
1	0	0	$A\bar{B}\bar{C}$ = m4
1	0	1	$A\bar{B}C$ = m5
1	1	0	$AB\bar{C}$ = m6
1	1	1	ABC = m7

Shorthand Notation for Minterms of 3 Variables



2-Level AND/OR Realization

F in canonical form:

$F(A,B,C) = \sum m(3,4,5,6,7)$

$= m3 + m4 + m5 + m6 + m7$

$$F = \bar{A}BC + A\bar{B}\bar{C}$$

canonical form ≠ minimal form

$$F = A\bar{B}(C + \bar{C}) + \bar{A}BC + AB(C + \bar{C})$$

$$= A\bar{B} + \bar{A}BC + AB$$

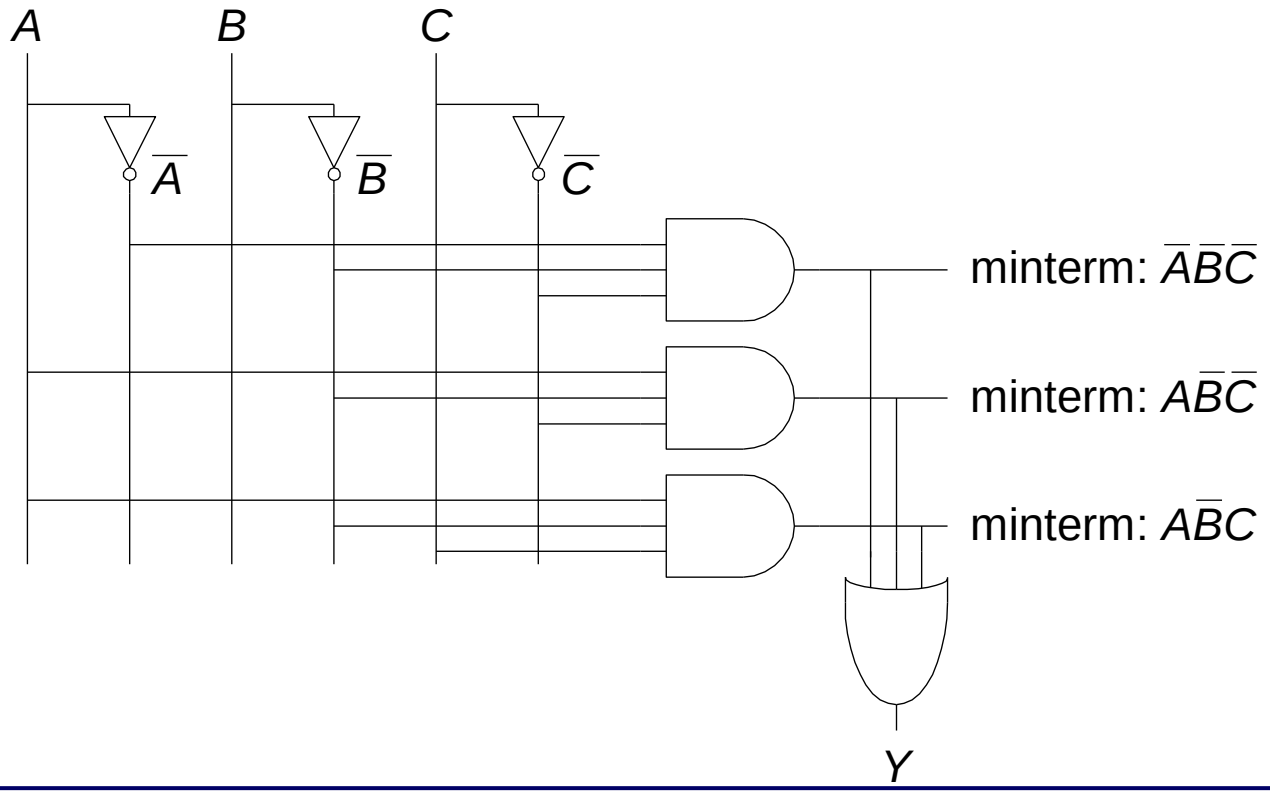
$$= A(\bar{B} + B) + \bar{A}BC$$

$$= A + \bar{A}BC$$

$$= A + BC$$

From Logic to Gates

- **SOP (sum-of-products) leads to two-level logic**
- Example: $Y = \left(\overline{A} \cdot \overline{B} \cdot \overline{C}\right) + \left(A \cdot \overline{B} \cdot \overline{C}\right) + \left(A \cdot \overline{B} \cdot C\right)$



Canonical Sum of Products Form: Key Idea

- Any 1-bit function can be represented as a Sum of Products
- A “Product” is the Boolean AND that includes ALL input variables of the function → minterm
- The 1-bit Output of the Function can be represented as
  - Sum (OR) of all minterms that lead to a 1 in the Output
- Logically
  - The function evaluates to TRUE (i.e., output is 1) if ANY of the Products (minterms) causes the Output to be 1
  - SOP form represents the function as the SUM (OR) of all Products (minterms) that cause the Output to be 1

Alternative Canonical Form: POS

We can have another from of representation

DeMorgan of SOP of $\overline{F}$

A product of sums (POS)

products

$$F = (A + B + C)(A + B + \overline{C})(A + \overline{B} + C)$$

sums

This input

Each sum term represents one of the "zeros" of the function

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$$F = \underset{0\ 0\ 0}{(A + B + C)}\ \underset{0\ 0\ 1}{(A + B + \overline{C})}\ \underset{0\ 1\ 0}{(A + \overline{B} + C)}$$

Activates this term

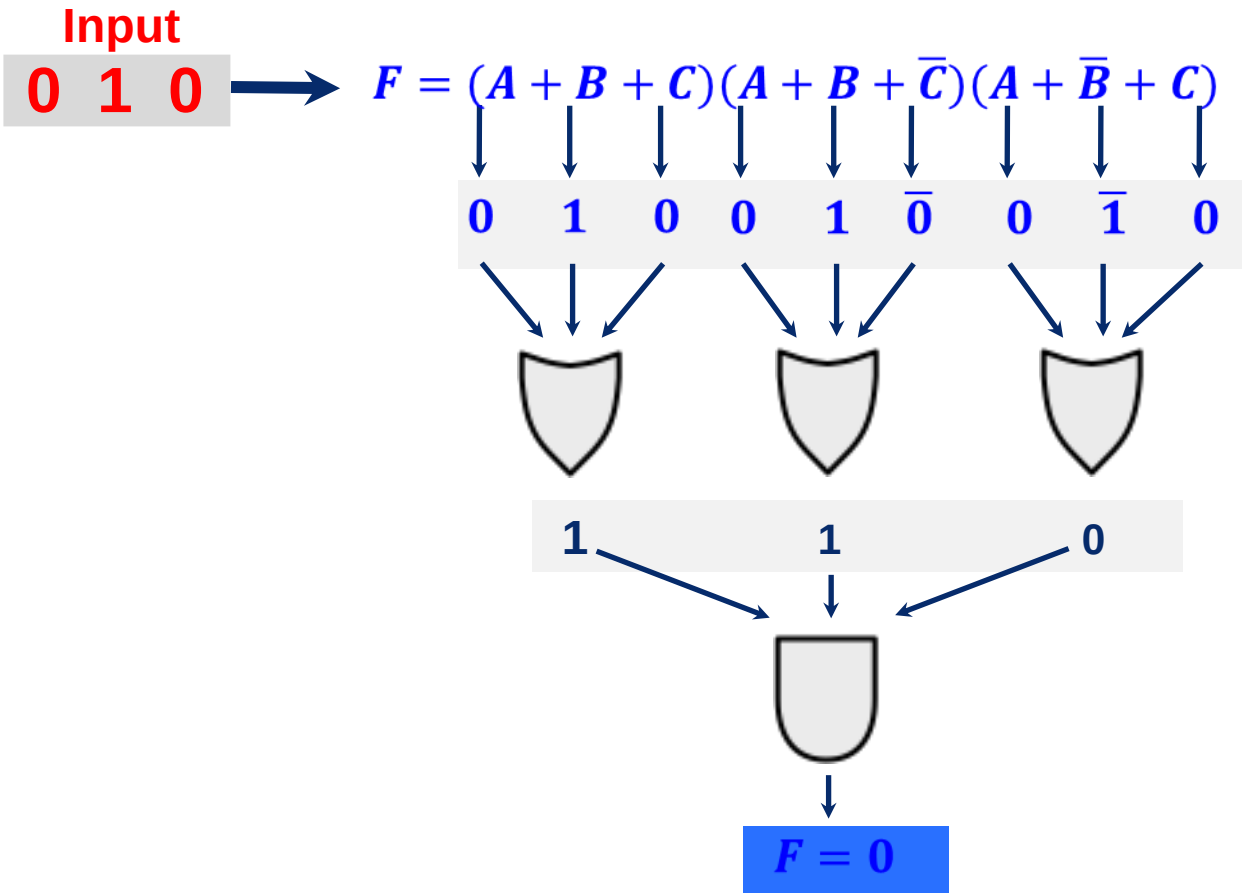
For the given input, only the shaded sum term will equal 0

$$A + \overline{B} + C = 0 + \overline{1} + 0$$

Anything ANDed with 0 is 0; Output F will be 0

Consider A=0, B=1, C=0

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1



Only one of the products will be 0, anything ANDed with 0 is 0

Therefore, the output is F = 0

POS: How to Write It

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$$F = (A + B + C)(A + B + \overline{C})(A + \overline{B} + C)$$

$$A \quad \overline{B} \quad C$$

$$A + \overline{B} + C$$

Maxterm form:

1. Find truth table rows where F is 0

2. 0 in input col → true literal
3. 1 in input col → complemented literal

4. OR the literals to get a Maxterm
5. AND together all the Maxterms

Or just remember, POS of F is the same as the DeMorgan of SOP of $\overline{F}$!!

Canonical POS Forms

Product of Sums / Conjunctive Normal Form / Maxterm Expansion

$$F = (A + B + C)(A + B + \bar{C})(A + \bar{B} + C)$$

$$\prod M(0, 1, 2)$$

A	B	C	Maxterms
0	0	0	$A + B + C$ = M0
0	0	1	$A + B + \bar{C}$ = M1
0	1	0	$A + \bar{B} + C$ = M2
0	1	1	$A + \bar{B} + \bar{C}$ = M3
1	0	0	$\bar{A} + B + C$ = M4
1	0	1	$\bar{A} + B + \bar{C}$ = M5
1	1	0	$\bar{A} + \bar{B} + C$ = M6
1	1	1	$\bar{A} + \bar{B} + \bar{C}$ = M7

Maxterm shorthand notation for a function of three variables

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Note that you form the maxterms around the "zeros" of the function

This is not the complement of the function!

Useful Conversions

1. **Minterm to Maxterm conversion:**
   rewrite minterm shorthand using maxterm shorthand
   replace minterm indices with the indices not already used
   E.g., $\mathrm{F}(A,B,C) = \sum m(3,4,5,6,7) = \prod M(0,1,2)$

2. **Maxterm to Minterm conversion:**
   rewrite maxterm shorthand using minterm shorthand
   replace maxterm indices with the indices not already used
   E.g., $\mathrm{F}(A,B,C) = \prod M(0,1,2) = \sum m(3,4,5,6,7)$

3. **Expansion of F to expansion of $\overline{F}$:**

$$\mathrm{E.g.,}\ \mathrm{F}(A,B,C) = \sum m(3,4,5,6,7) \longrightarrow \overline{F}(A,B,C) = \sum m(0,1,2)$$
$$= \prod M(0,1,2) \longrightarrow = \prod M(3,4,5,6,7)$$

4. **Minterm expansion of to Maxterm expansion of $\overline{F}$:**
   rewrite in Maxterm form, using the same indices as F

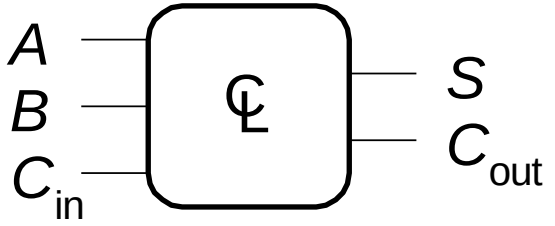
$$\mathrm{E.g.,}\ \mathrm{F}(A,B,C) = \sum m(3,4,5,6,7) \longrightarrow \overline{F}(A,B,C) = \prod M(3,4,5,6,7)$$
$$= \prod M(0,1,2) \longrightarrow = \sum m(0,1,2)$$

Logic Simplification (or Minimization)

- Using Boolean Algebra, we can simplify the SOP or POS form of any function in a methodical way
- Starting with the canonical SOP or POS form enables convenience and automation
  - Truth table → SOP/POS form → Boolean Simplification Rules

- ***Example (full 1-bit adder – more later):***

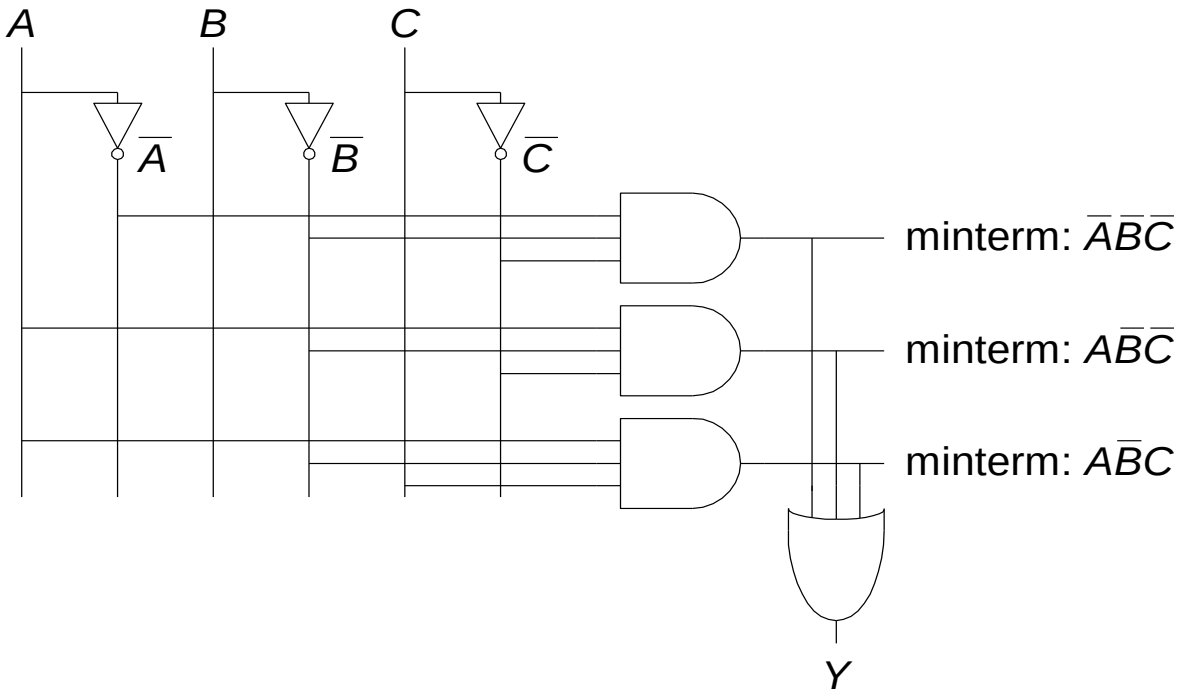
$$S = F(A, B, C_{in})$$
$$C_{out} = G(A, B, C_{in})$$



$$S = A \oplus B \oplus C_{in}$$
$$C_{out} = AB + AC_{in} + BC_{in}$$

Logic Simplification Example: SOP Form

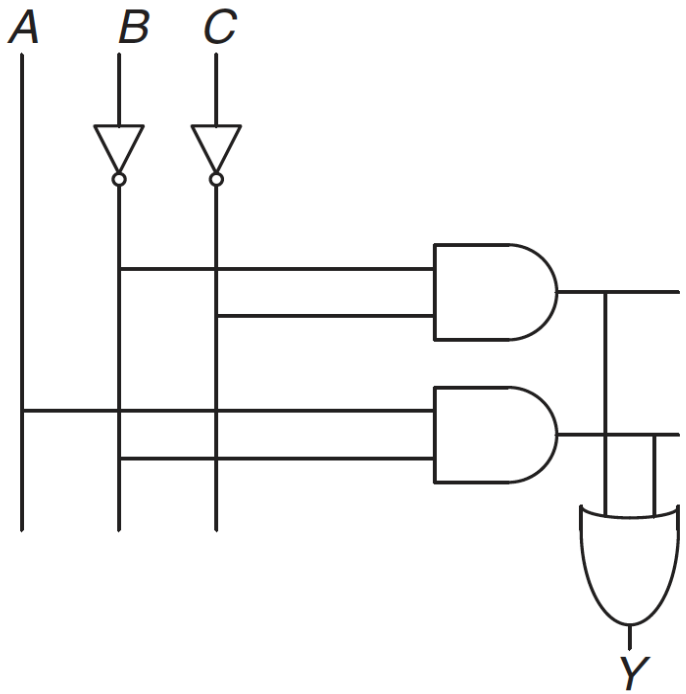
- **SOP (sum-of-products) form of function Y**
- Example: $Y = \left(\overline{A} \cdot \overline{B} \cdot \overline{C}\right) + \left(A \cdot \overline{B} \cdot \overline{C}\right) + \left(A \cdot \overline{B} \cdot C\right)$



Logic Simplification Example: Simplified

- **SOP (sum-of-products) form of function Y**

- Example: $Y = \left(\overline{B} \cdot \overline{C}\right) + \left(A \cdot \overline{B}\right)$



Let's Cover Some Basic Combinational Blocks

Combinational Building Blocks used in Modern Computers

Combinational Building Blocks

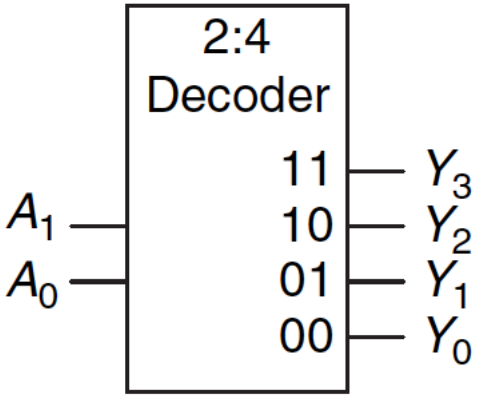
- Combinational logic is often grouped into larger building blocks to build more complex systems
- Hides the unnecessary gate-level details to emphasize the function of the building block
- We now examine:
  - Decoder
  - Multiplexer
  - Full adder
  - PLA (Programmable Logic Array)

Decoder

Decoder

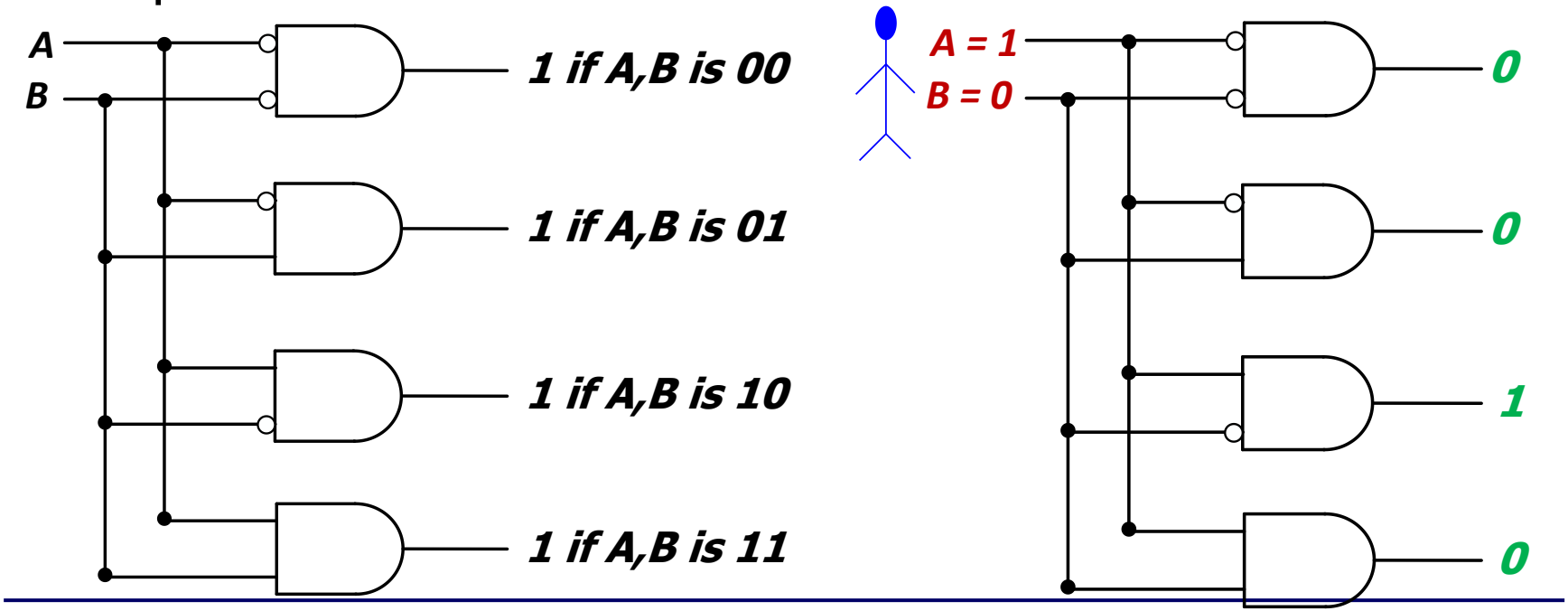
- "Input pattern detector"
- n inputs and 2^n outputs
- Exactly one of the outputs is 1 and all the rest are 0s
- The one output that is logically 1 is the output corresponding to the input pattern that the logic circuit is expected to detect
- Example: 2-to-4 decoder

A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



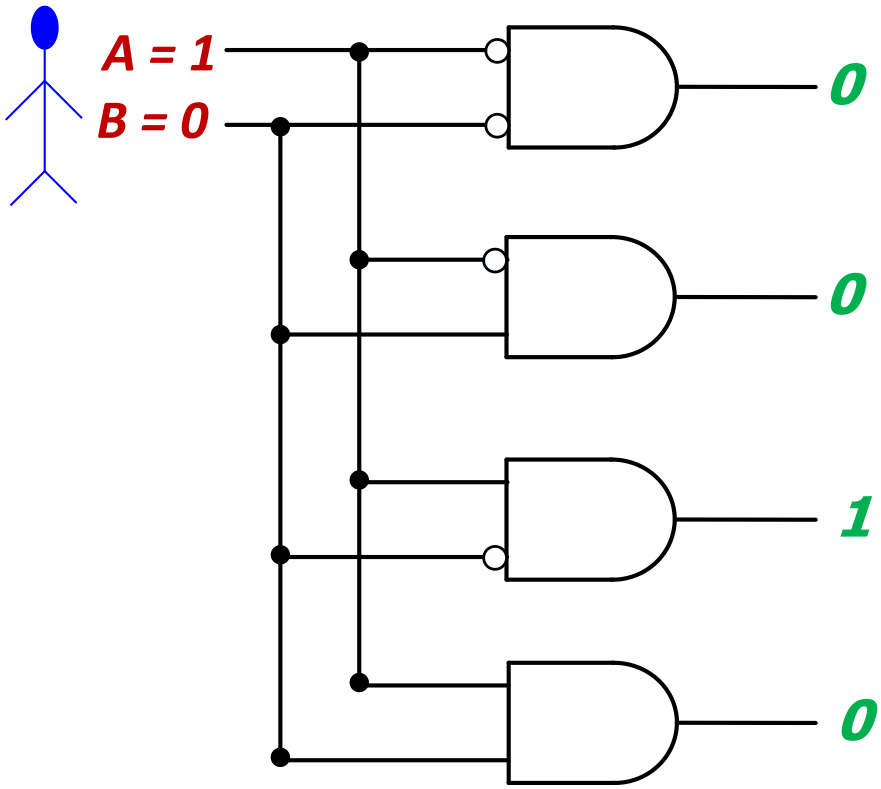
Decoder (I)

- n inputs and 2^n outputs
- Exactly one of the outputs is 1 and all the rest are 0s
- The one output that is logically 1 is the output corresponding to the input pattern that the logic circuit is expected to detect



Decoder (II)

- The decoder is useful in determining how to interpret a bit pattern
  - **It could be the address of a row in DRAM, that the processor intends to read from**
  - **It could be an instruction in the program and the processor needs to decide what action to take (based on *instruction opcode*)**

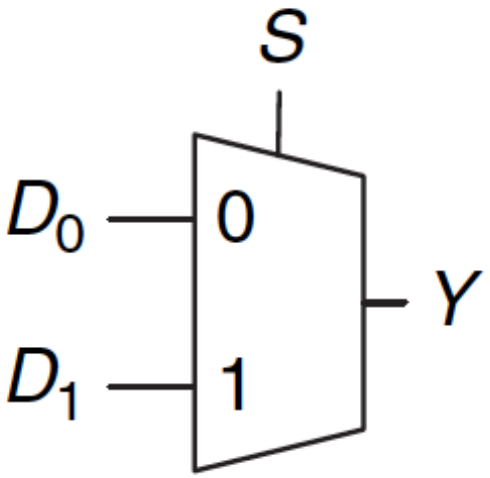


Multiplexer (MUX)

Multiplexer (MUX), or Selector

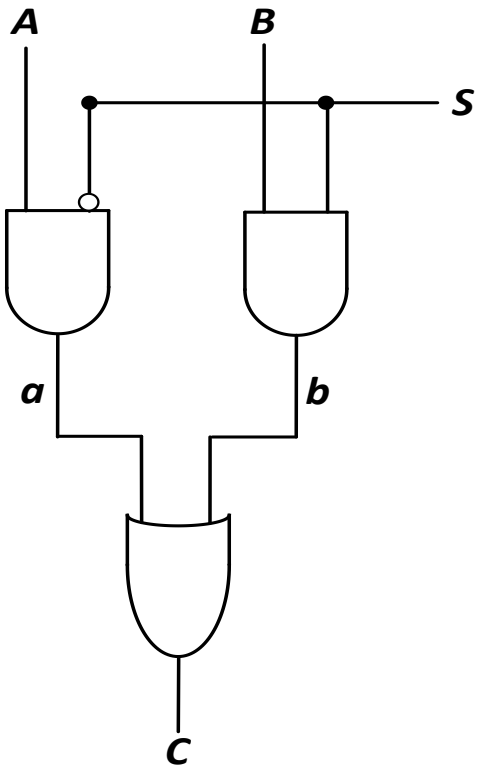
- Selects one of the *N* inputs to connect it to the output
  - based on the value of a $\log_2 N$-bit control input called select
- Example: 2-to-1 MUX

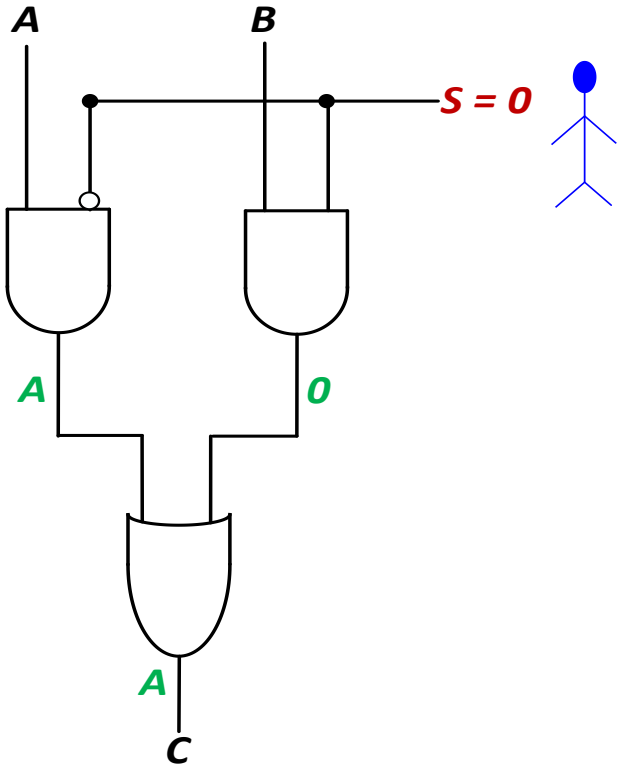
S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



Multiplexer (MUX), or Selector (II)

- Selects one of the *N* inputs to connect it to the output
  - based on the value of a $\log_2 N$-bit control input called select
- Example: 2-to-1 MUX

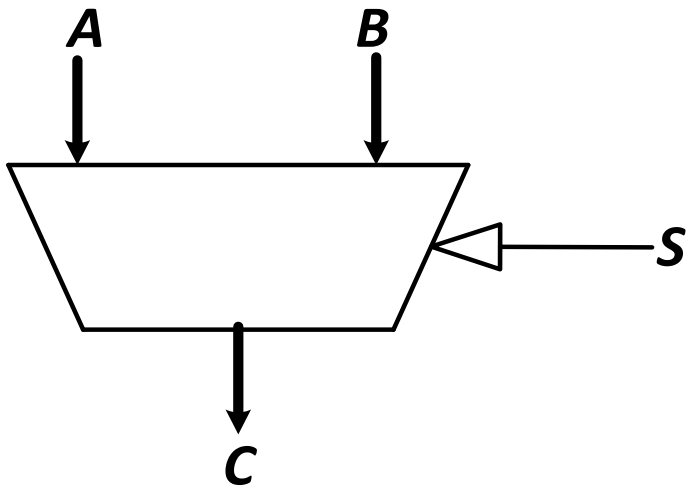




Multiplexer (MUX), or Selector (III)

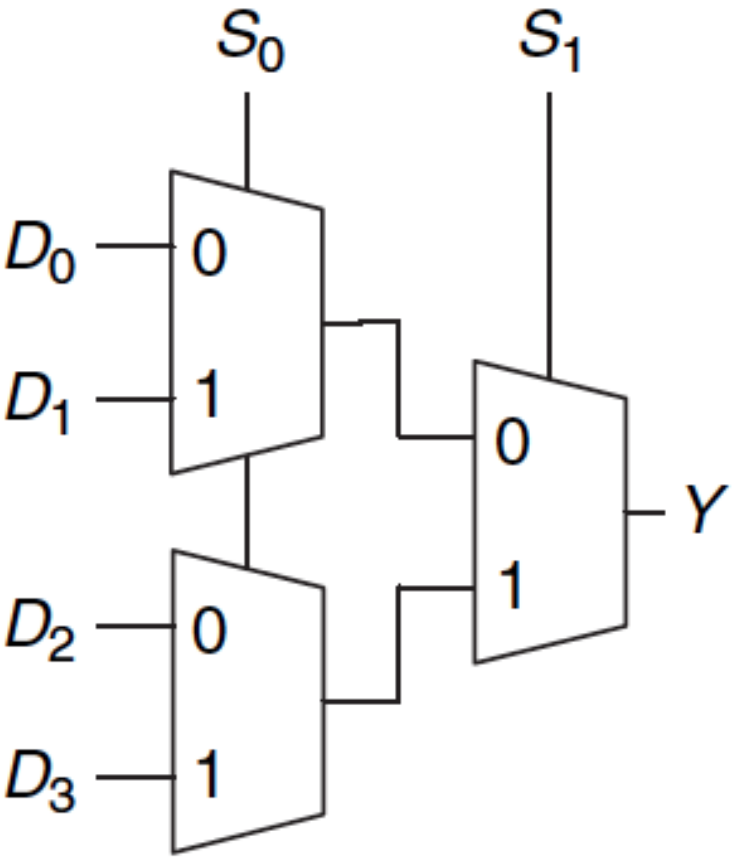
- The output C is always connected to either the input A or the input B
  - Output value depends on the value of the select line S

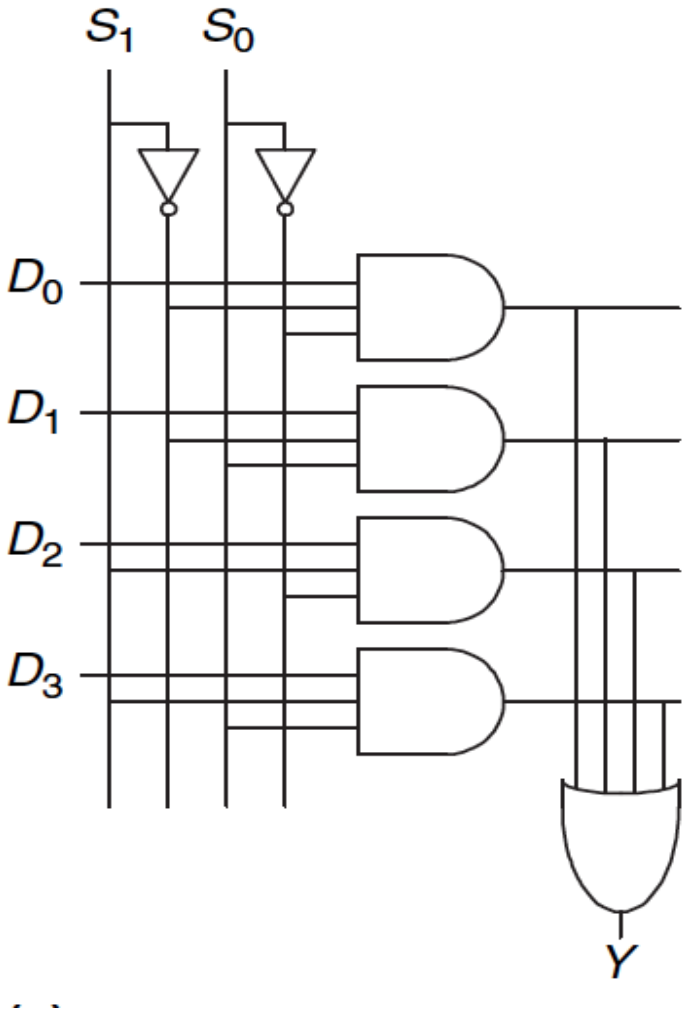
S	C
0	A
1	B



- Your task: Draw the schematic for an 4-input (4:1) MUX
  - Gate level: as a combination of basic AND, OR, NOT gates
  - Module level: As a combination of 2-input (2:1) MUXes

A 4-to-1 Multiplexer





Full Adder

Full Adder (I)

- **Binary addition**
  - Similar to decimal addition
  - From right to left
  - One column at a time
  - One sum and one carry bit

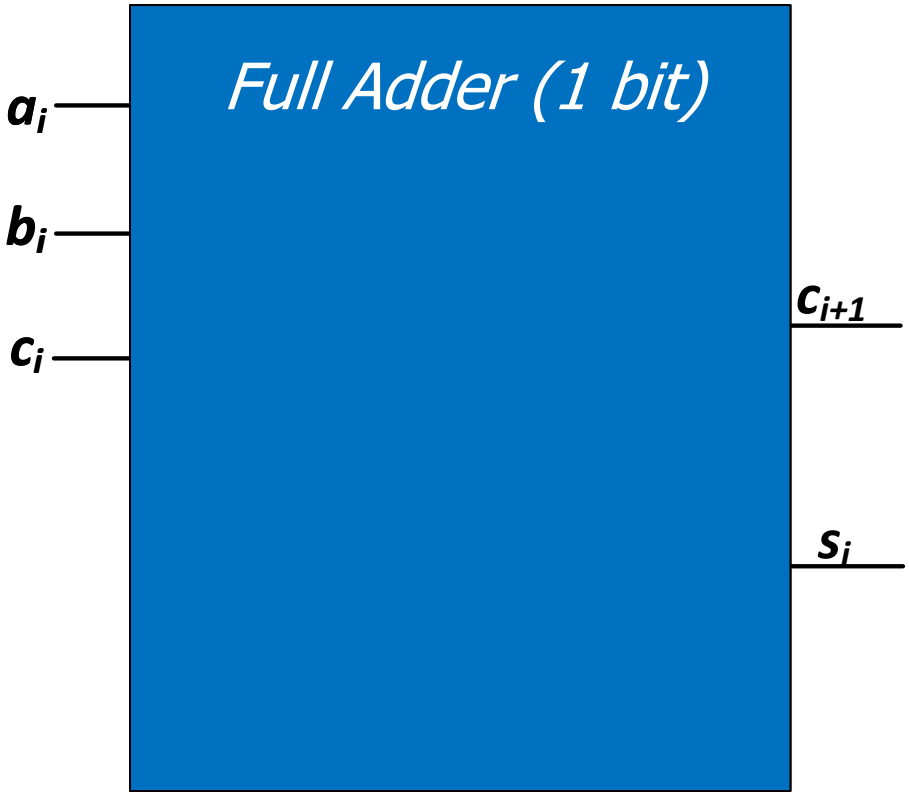
$$\begin{array}{r} a_{n-1}a_{n-2}\ldots a_1a_0 \\ b_{n-1}b_{n-2}\ldots b_1b_0 \\ C_n\,C_{n-1}\quad\ldots\quad C_1 \\ \hline S_{n-1}\quad\ldots\quad S_1S_0 \end{array}$$

- Truth table of binary addition on one column of bits within two n-bit operands

a_i	b_i	$carry_i$	$carry_{i+1}$	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Full Adder (II)

- **Binary addition**
  - N 1-bit additions
  - **SOP of 1-bit addition**

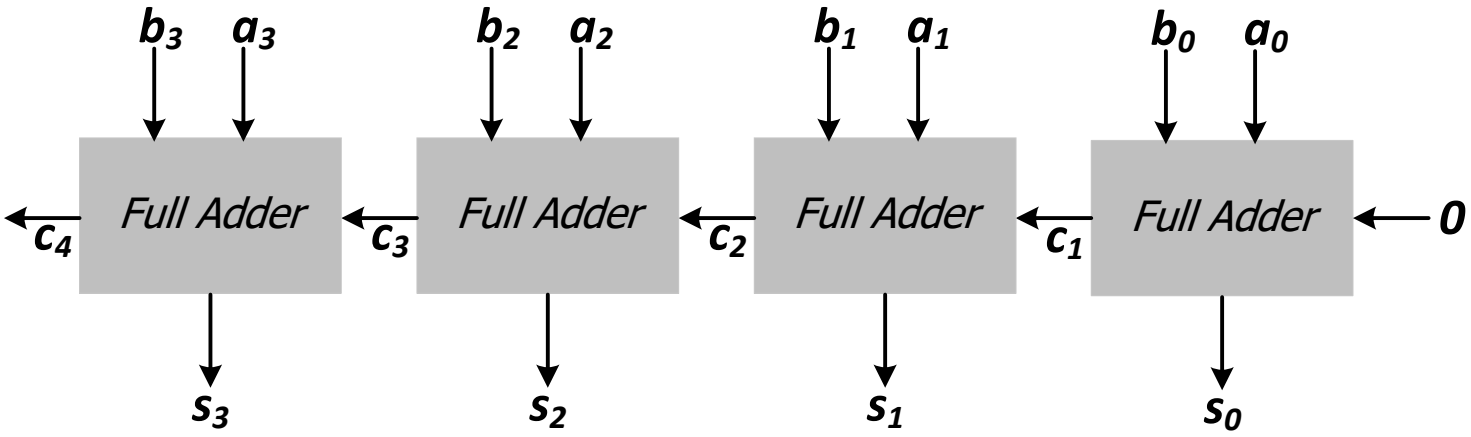


$$a_{n-1}a_{n-2}\ldots a_1a_0$$
$$b_{n-1}b_{n-2}\ldots b_1b_0$$
$$C_n\ C_{n-1}\quad\ldots\quad C_1$$
$$S_{n-1}\quad\ldots\quad S_1S_0$$

a_i	b_i	$carry_i$	$carry_{i+1}$	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

4-Bit Adder from Full Adders

- Creating a **4-bit adder** out of 1-bit full adders
  - To add two 4-bit binary numbers A and B



$$\begin{array}{cccc} & a_3 & a_2 & a_1 & a_0 \\ + & b_3 & b_2 & b_1 & b_0 \\ \hline c_4 & c_3 & c_2 & c_1 & \\ \hline & s_3 & s_2 & s_1 & s_0 \end{array} \qquad \begin{array}{cccc} & 1 & 0 & 1 & 1 \\ + & 1 & 0 & 0 & 1 \\ \hline 1 & 0 & 1 & 1 & \\ \hline & 0 & 1 & 0 & 0 \end{array}$$

Adder Design: Ripple Carry Adder

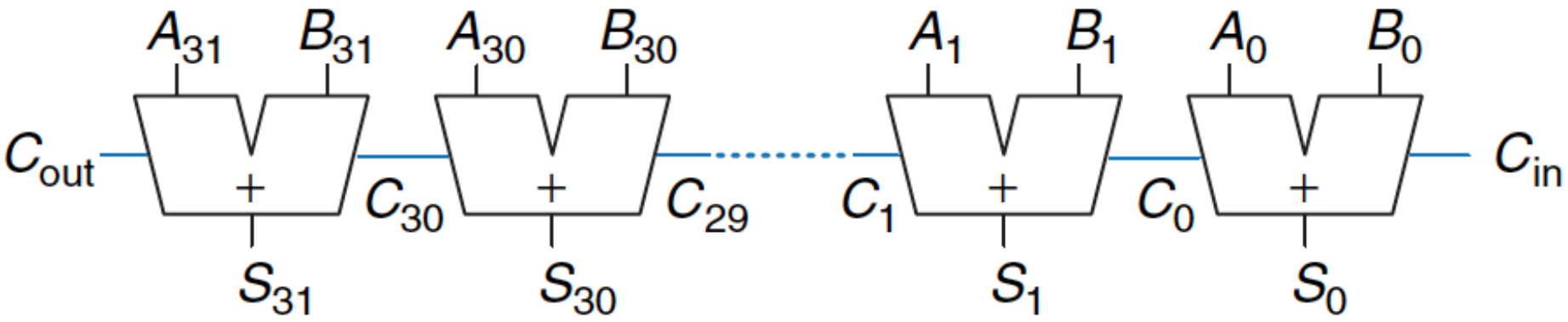
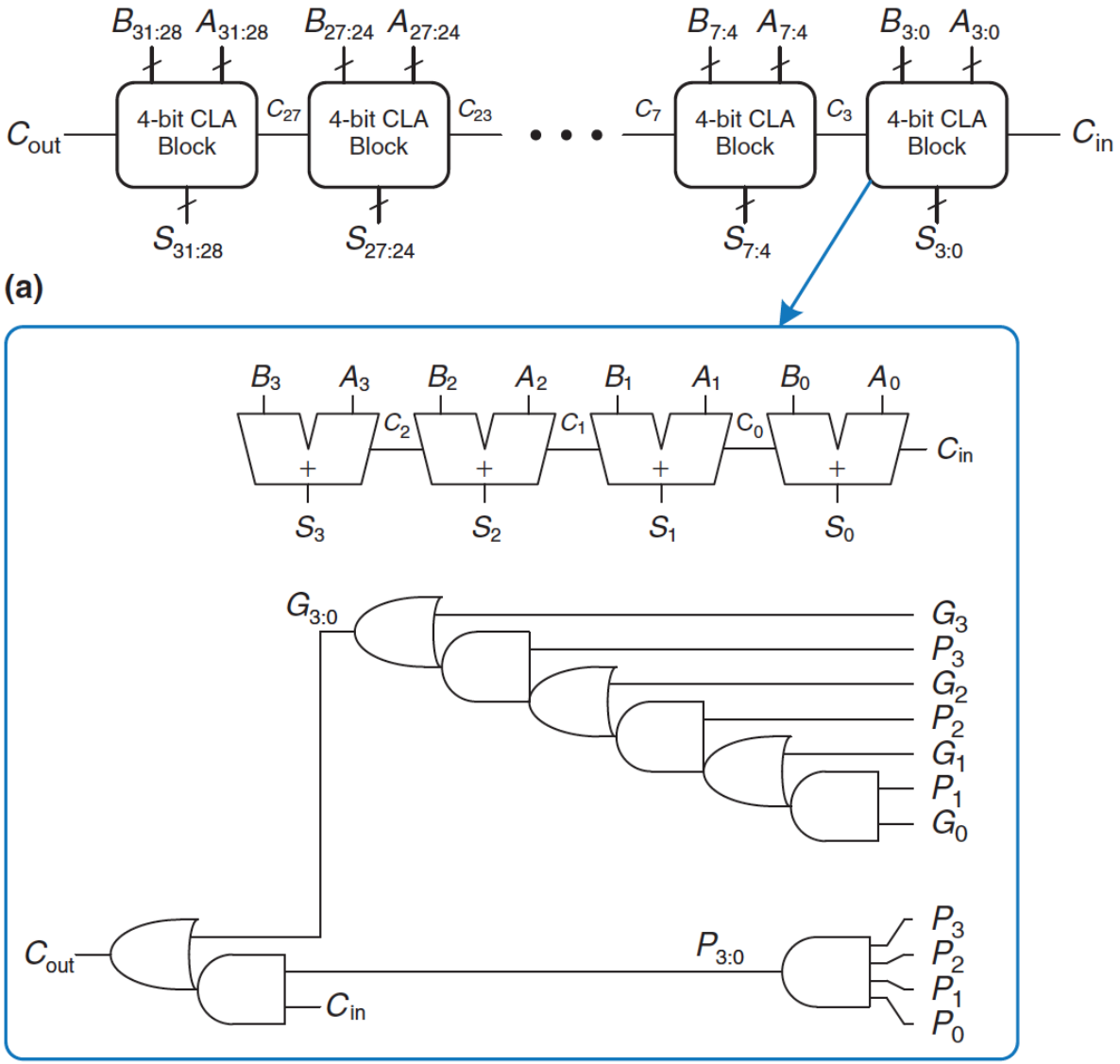


Figure 5.5 32-bit ripple-carry adder

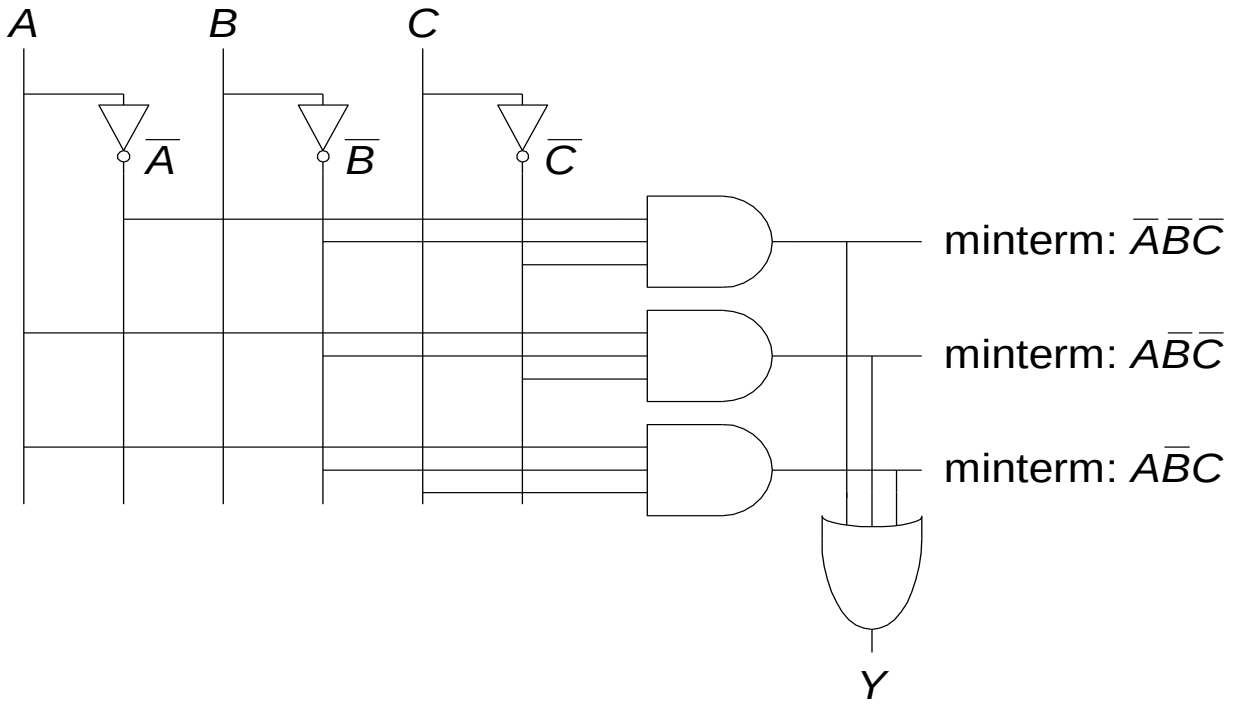
Adder Design: Carry Lookahead Adder



Programmable Logic Array (PLA)

PLA: Recall: From Logic to Gates

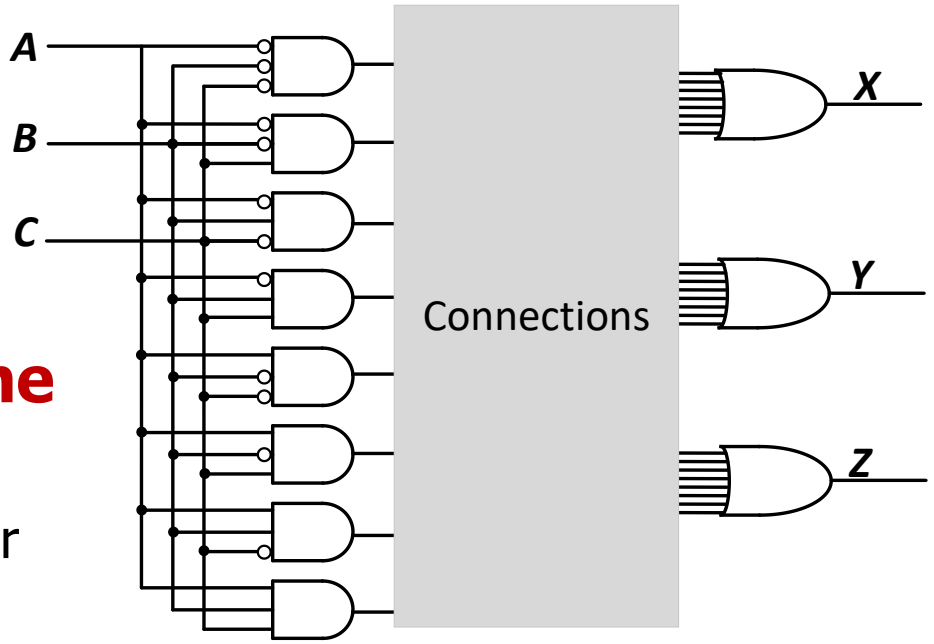
- **SOP (sum-of-products) leads to two-level logic**
- Example: $Y = \left(\overline{A} \cdot \overline{B} \cdot \overline{C}\right) + \left(A \cdot \overline{B} \cdot \overline{C}\right) + \left(A \cdot \overline{B} \cdot C\right)$



A PLA enables the two-level SOP implementation of **any** N-input M-output function

The Programmable Logic Array (PLA)

- The below logic structure is a very common building block for implementing any collection of logic functions one wishes to

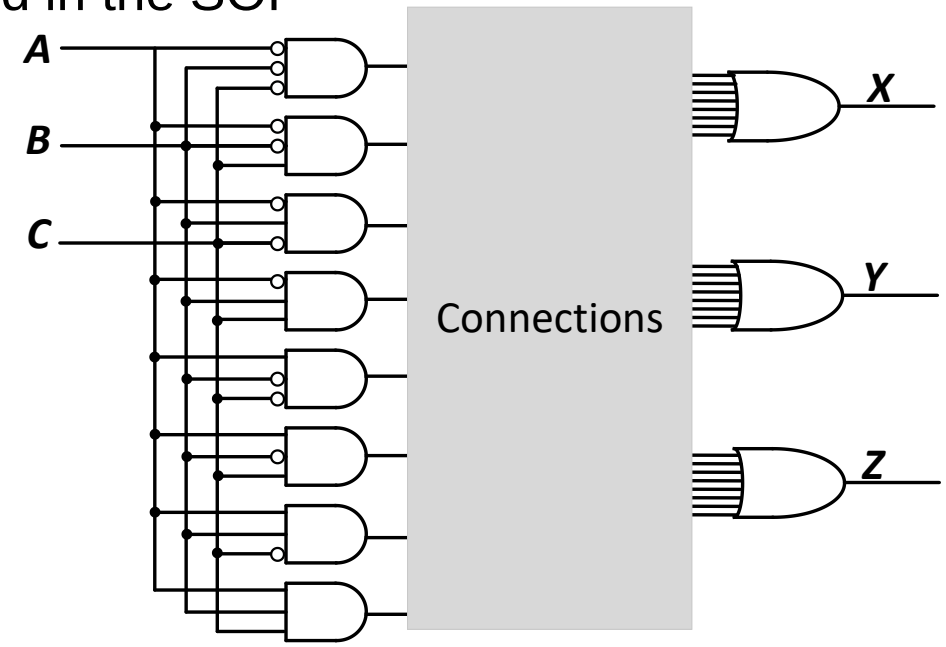


- An array of AND gates followed by an array of OR gates
- **How do we determine the number of AND gates?**
  - **Remember SOP:** the number of possible minterms
  - For an n-input logic function, we need a PLA with 2^n n-input AND gates
- **How do we determine the number of OR gates?** The number of output columns in the truth table

A PLA enables the two-level SOP implementation of **any** N-input M-output function

The Programmable Logic Array (PLA)

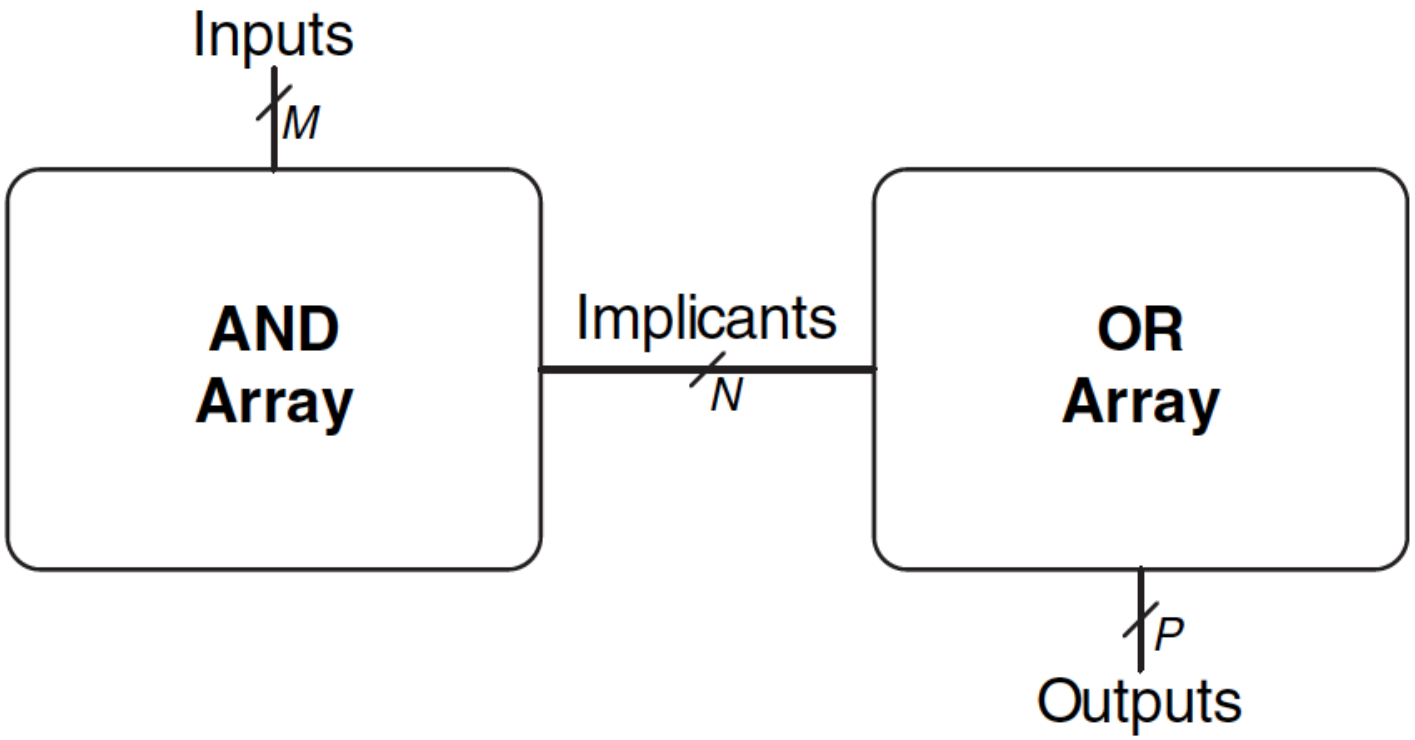
- How do we implement a logic function?
  - Connect the output of an AND gate to the input of an OR gate if the corresponding minterm is included in the SOP
  - This is a simple programmable logic construct



- **Programming a PLA**: we program the connections from AND gate outputs to OR gate inputs to implement a desired logic function
- Have you seen any other type of programmable logic?
  - Yes! An FPGA…
  - An FPGA uses more advanced structures, as we saw in Lecture 3

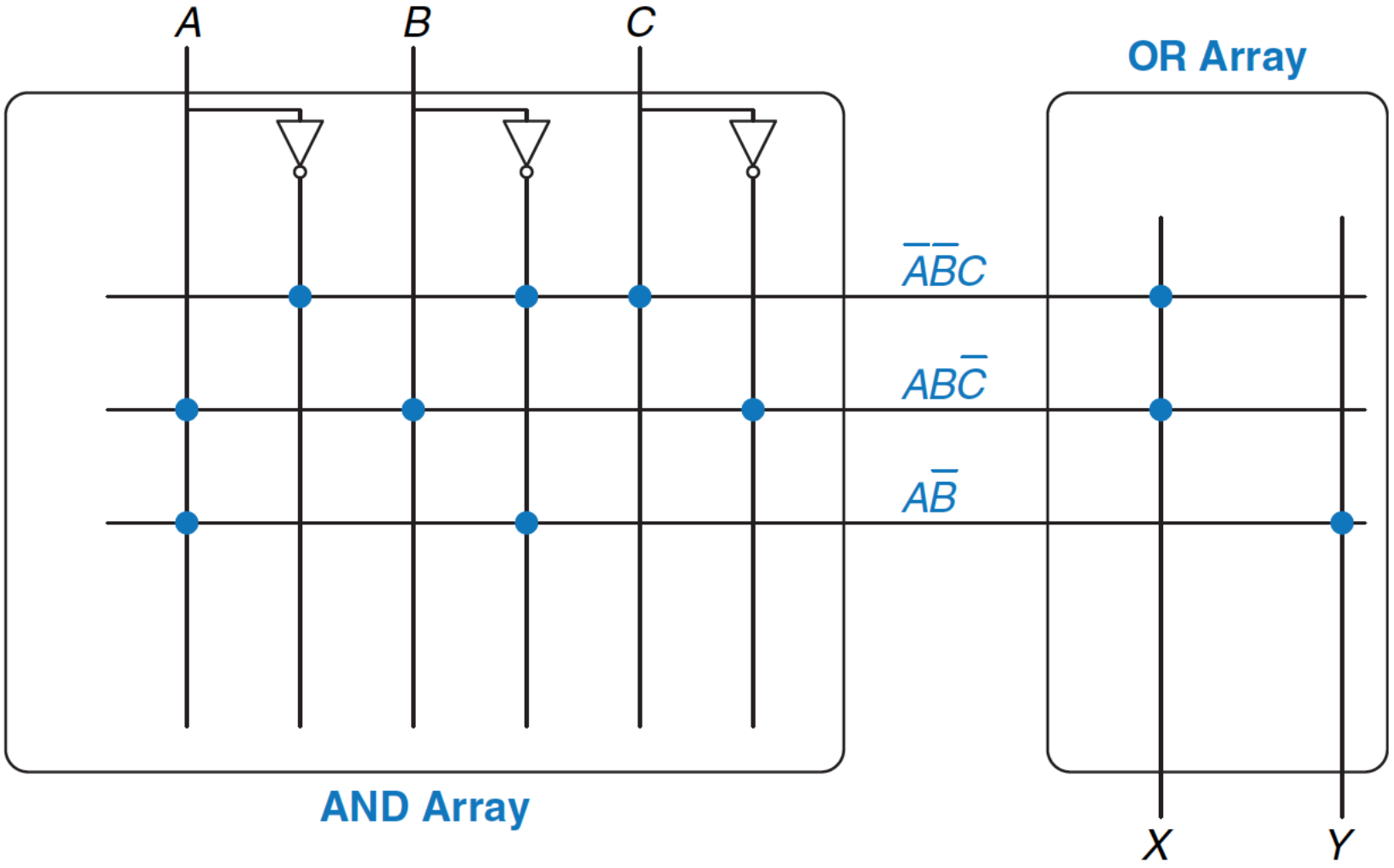
A PLA enables the two-level SOP implementation of **any** N-input M-output function

PLA Example (I)



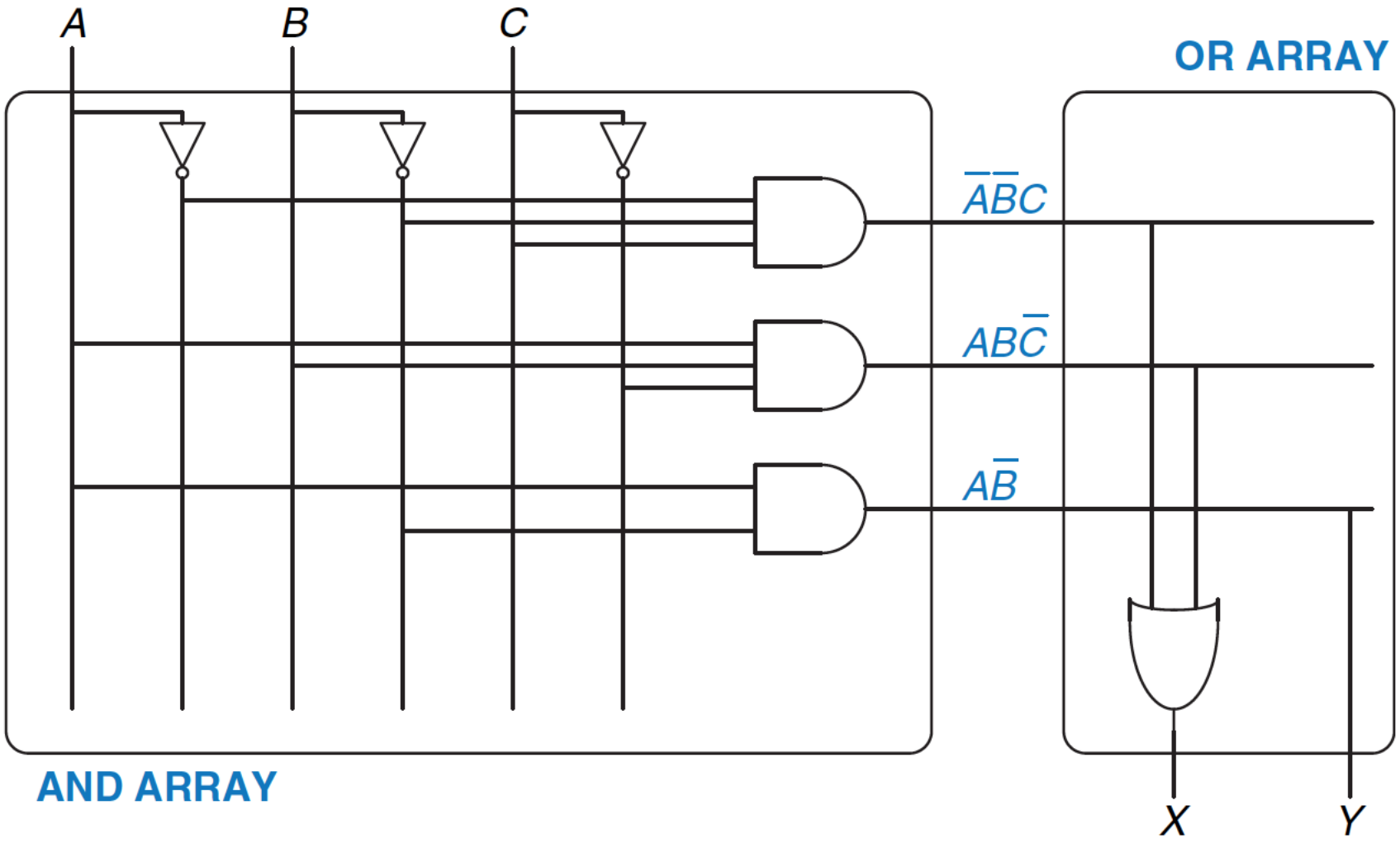
Read H&H Chapter 5.6.1

PLA Example Function (II)



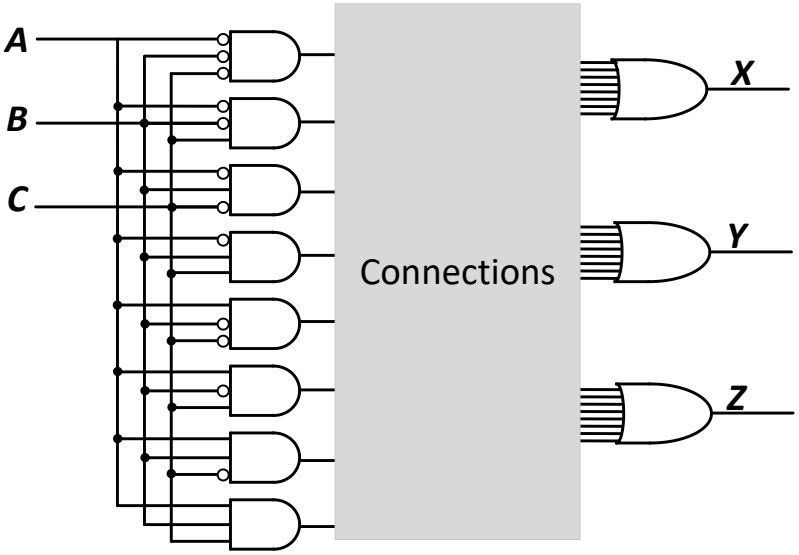
Read H&H Chapter 5.6.1

PLA Example Function (III)



Read H&H Chapter 5.6.1

Implementing a Full Adder Using a PLA

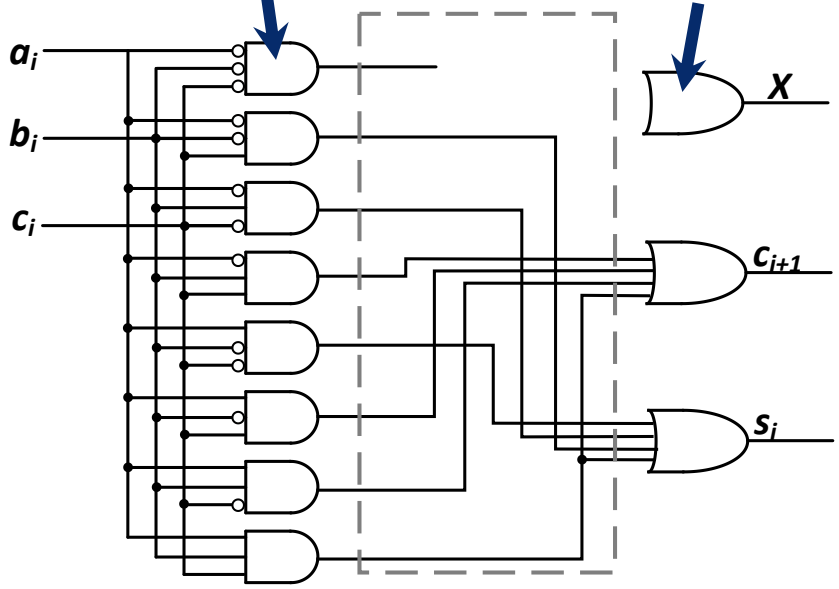


Truth table of a full adder

a_i	b_i	$carry_i$	$carry_{i+1}$	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

This input should not be connected to any outputs

We do not need this output



Logical Completeness

Logical (Functional) Completeness

- Any logic function we wish to implement could be accomplished with a PLA
  - PLA consists of only AND gates, OR gates, and inverters
  - We just have to program connections based on SOP of the intended logic function

- The set of gates {AND, OR, NOT} is logically complete because we can build a circuit to carry out the specification of any truth table we wish, without using any other kind of gate

- NAND is also logically complete. So is NOR.
  - Your task: Prove this.

More Combinational Blocks

© Osmonov M.S.
IUCA

More Combinational Building Blocks

- H&H Chapter 2 in full
  - Required Reading
  - E.g., see Tri-state Buffer and Z values in Section 2.6

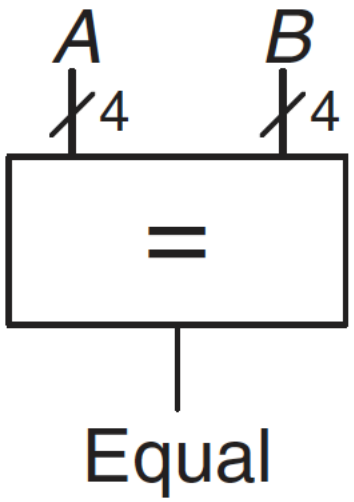
- H&H Chapter 5
  - Will be required reading soon.

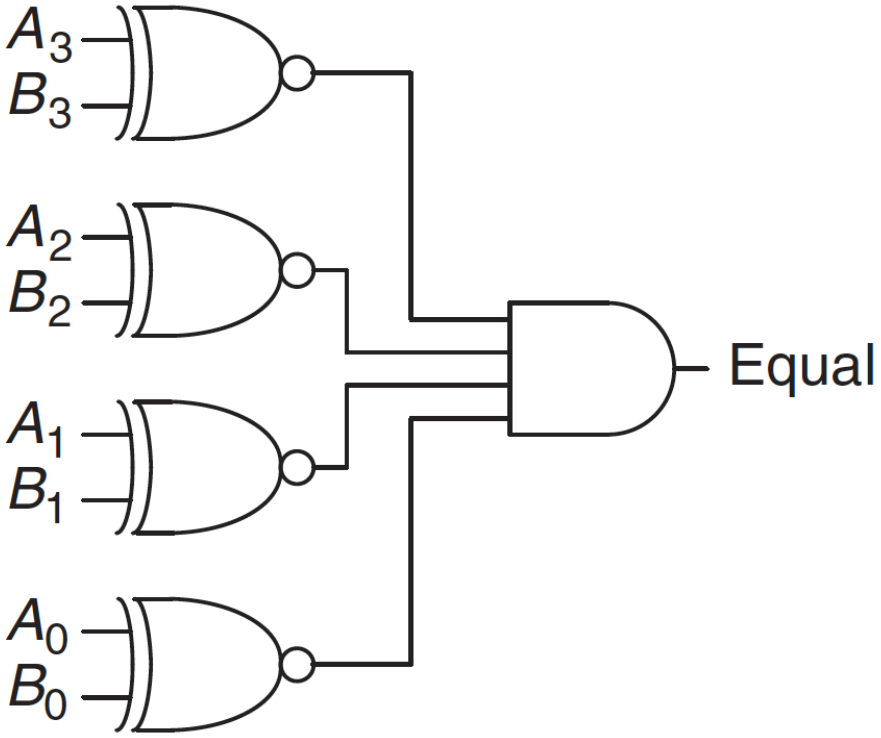
- You will benefit greatly by reading the “combinational” parts of Chapter 5 soon.
  - Sections 5.1 and 5.2

Comparator

Equality Checker (Compare if Equal)

- Checks if two N-input values are exactly the same
- Example: 4-bit Comparator





ALU
(Arithmetic Logic Unit)

ALU (Arithmetic Logic Unit)

- Combines a variety of arithmetic and logical operations into a single unit (that performs only one function at a time)
- Usually denoted with this symbol:

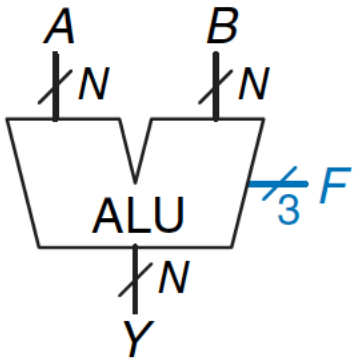


Figure 5.14 ALU symbol

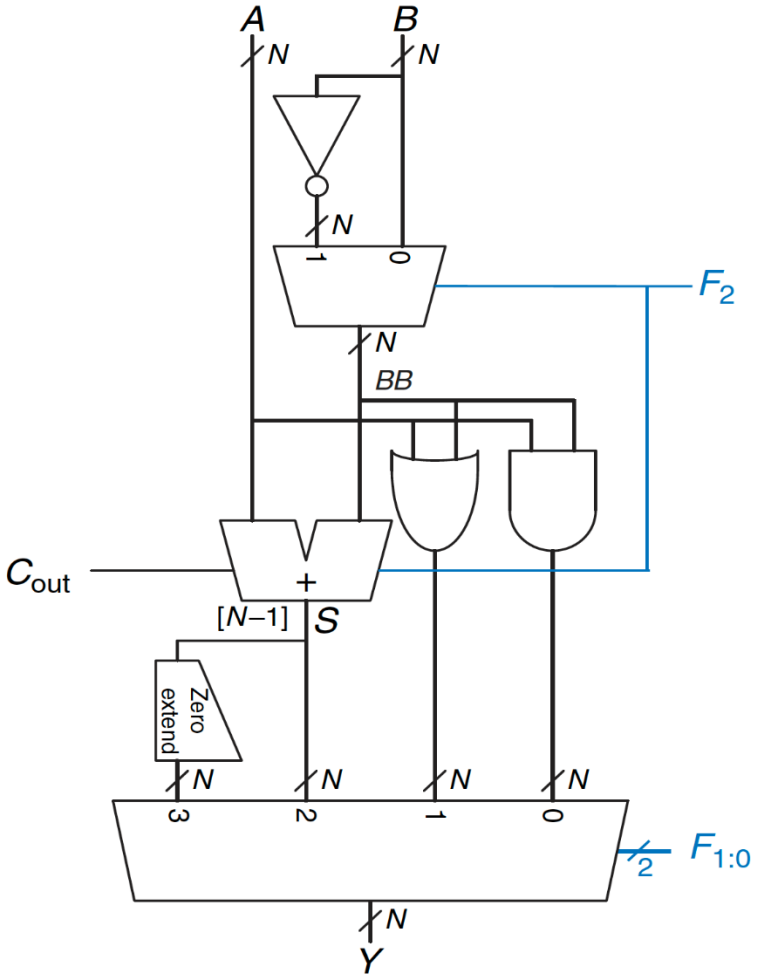
Table 5.1 ALU operations

$F_{2:0}$	Function
000	A AND B
001	A OR B
010	A + B
011	not used
100	A AND $\overline{B}$
101	A OR $\overline{B}$
110	A – B
111	SLT

Example ALU (Arithmetic Logic Unit)

Table 5.1 ALU operations

$F_{2:0}$	Function
000	A AND B
001	A OR B
010	A + B
011	not used
100	A AND $\overline{B}$
101	A OR $\overline{B}$
110	A – B
111	SLT



More Combinational Building Blocks

- See H&H Chapter 5.2 for
  - Subtractor (using 2's Complement Representation)
  - Shifter and Rotator
  - Multiplier
  - Divider
  - …

More Combinational Building Blocks

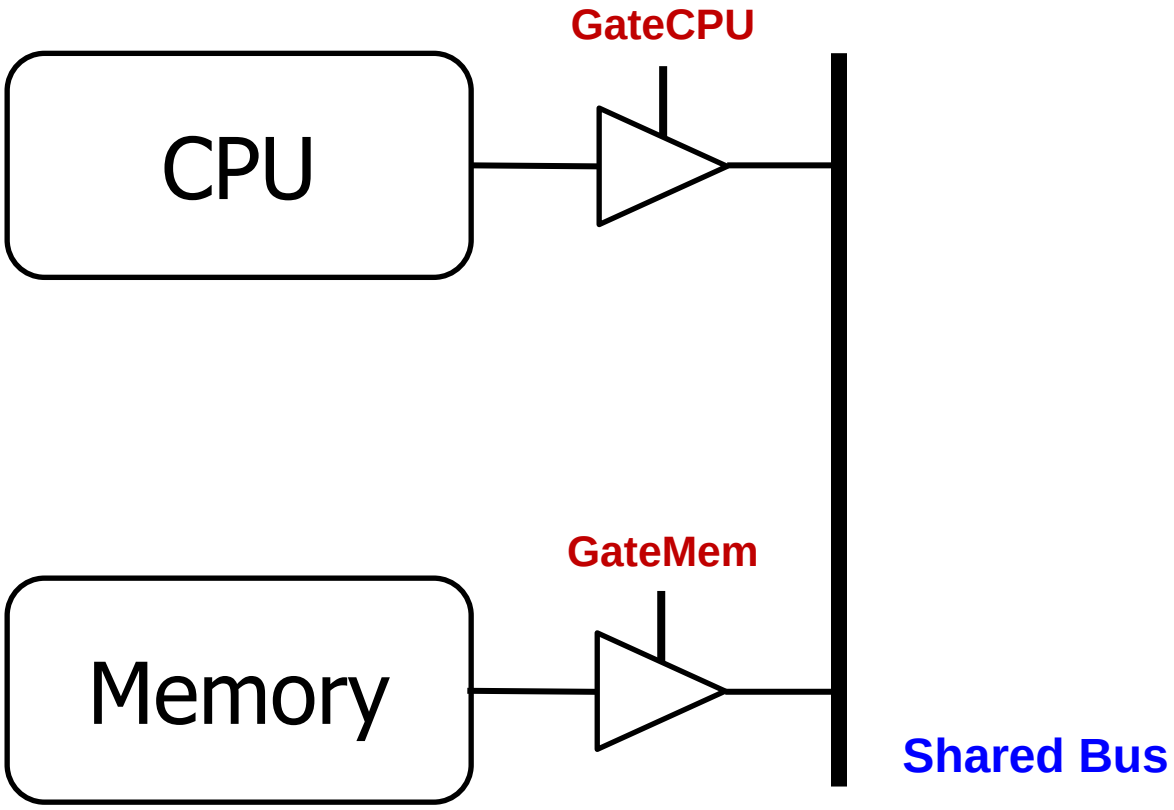
- H&H Chapter 2 in full
  - Required Reading
  - E.g., see Tri-state Buffer and Z values in Section 2.6
- H&H Chapter 5
  - Will be required reading soon.
- You will benefit greatly by reading the “combinational” parts of Chapter 5 soon.
  - Sections 5.1 and 5.2

Tri-State Buffer

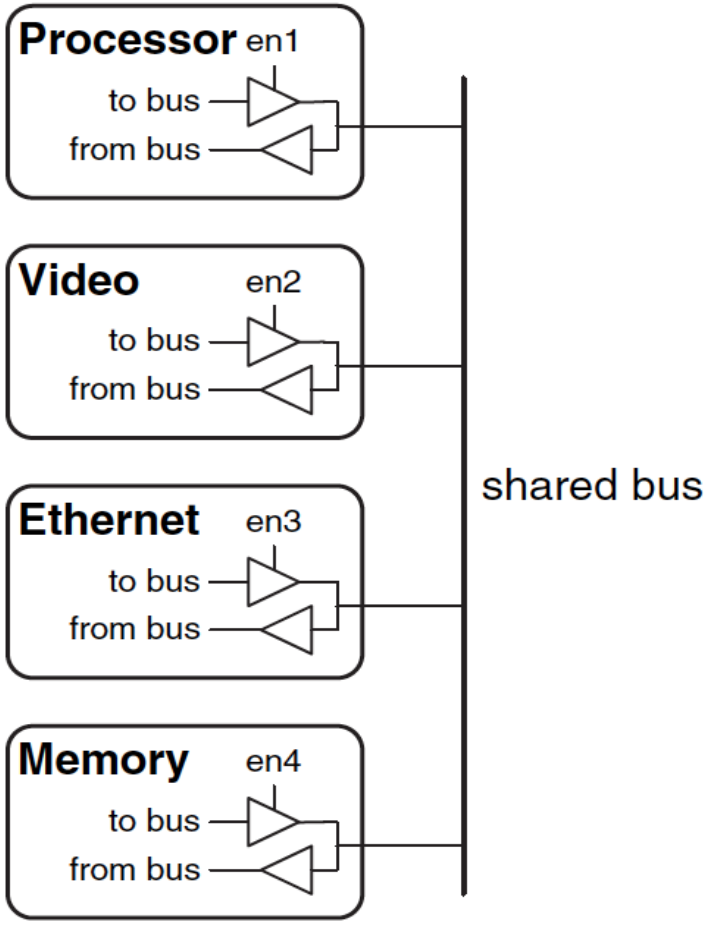
Example: Use of Tri-State Buffers

- Imagine a wire connecting the CPU and memory
  - At any time only the CPU or the memory can place a value on the wire, both not both
  - You can have two tri-state buffers: one driven by CPU, the other memory; and ensure at most one is enabled at any time

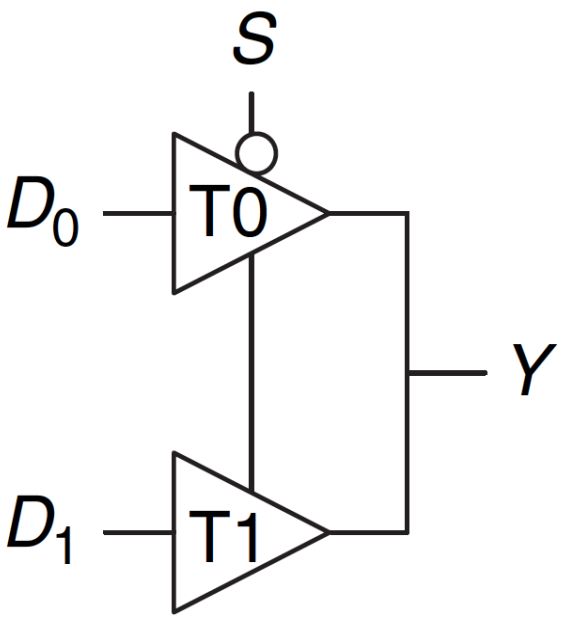
Example Design with Tri-State Buffers



Another Example



Multiplexer Using Tri-State Buffers



$$Y = D_0\overline{S} + D_1 S$$

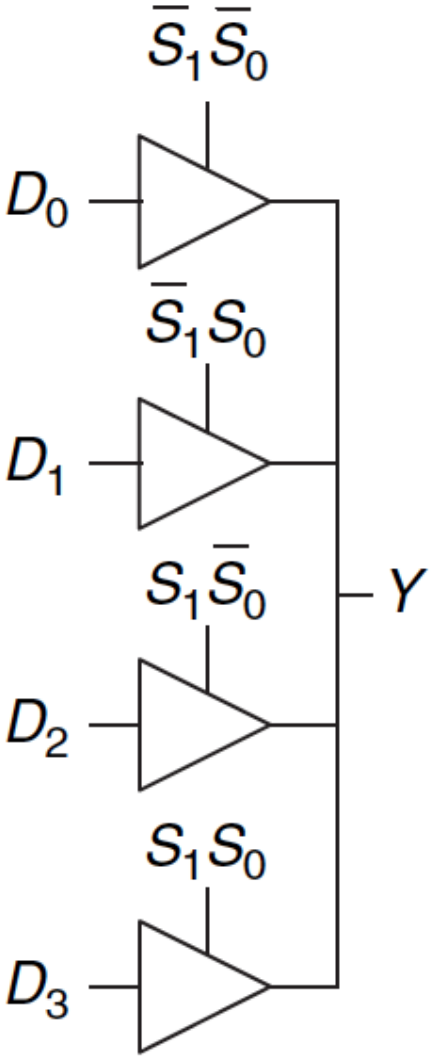


Figure 2.56 Multiplexer using tristate buffers

Aside: Logic Using Multiplexers

- Multiplexers can be used as lookup tables to perform logic functions

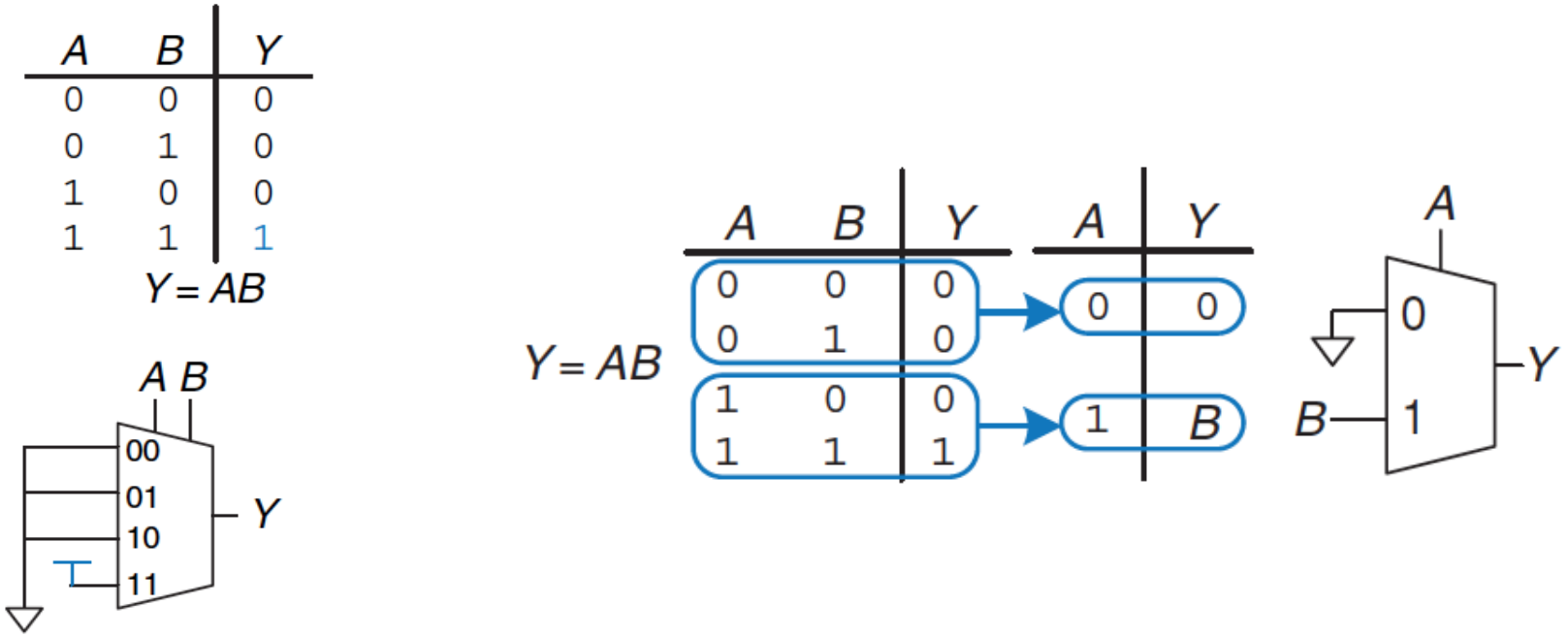
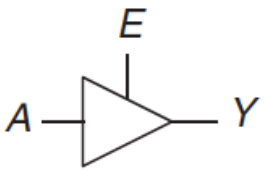


Figure 2.59 **4:1 multiplexer implementation of two-input AND function**

Tri-State Buffer

- A tri-state buffer enables gating of different signals onto a wire

Tristate Buffer



E	A	Y
0	0	Z
0	1	Z
1	0	0
1	1	1

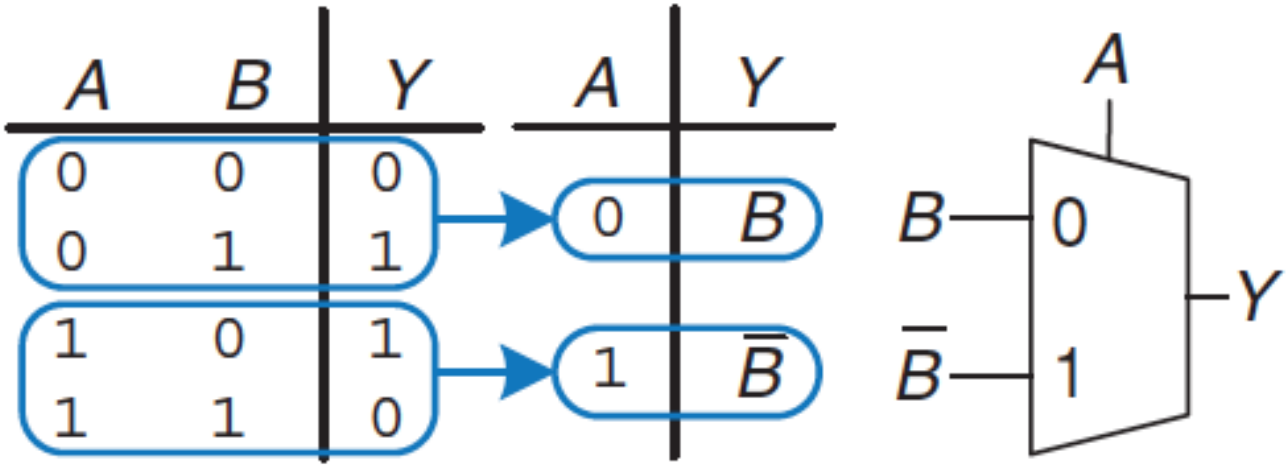
Figure 2.40 Tristate buffer

- Floating signal (Z): Signal that is not driven by any circuit
  - Open circuit, floating wire

Aside: Logic Using Multiplexers (II)

- Multiplexers can be used as lookup tables to perform logic functions

$Y = A \oplus B$

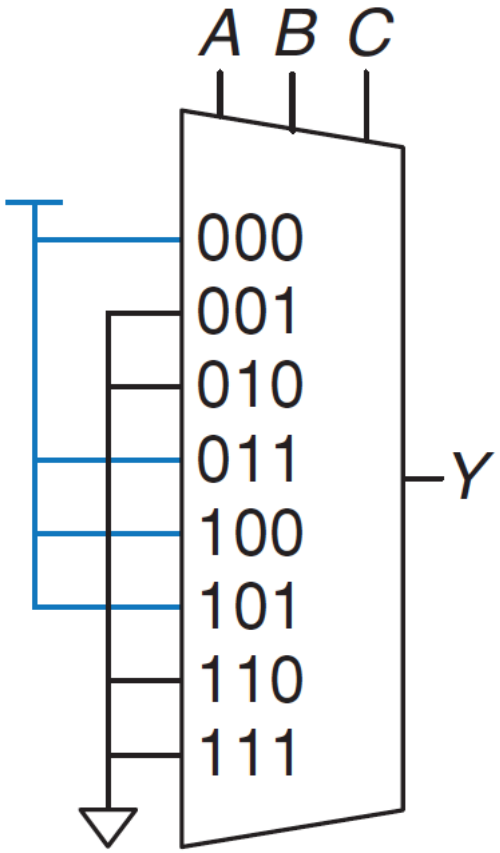


Aside: Logic Using Multiplexers (III)

- Multiplexers can be used as lookup tables to perform logic functions

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

$$Y = A\overline{B} + \overline{B}\overline{C} + \overline{A}BC$$



Read H&H Chapter 2.8

Aside: Logic Using Decoders (I)

- Decoders can be combined with OR gates to build logic functions.

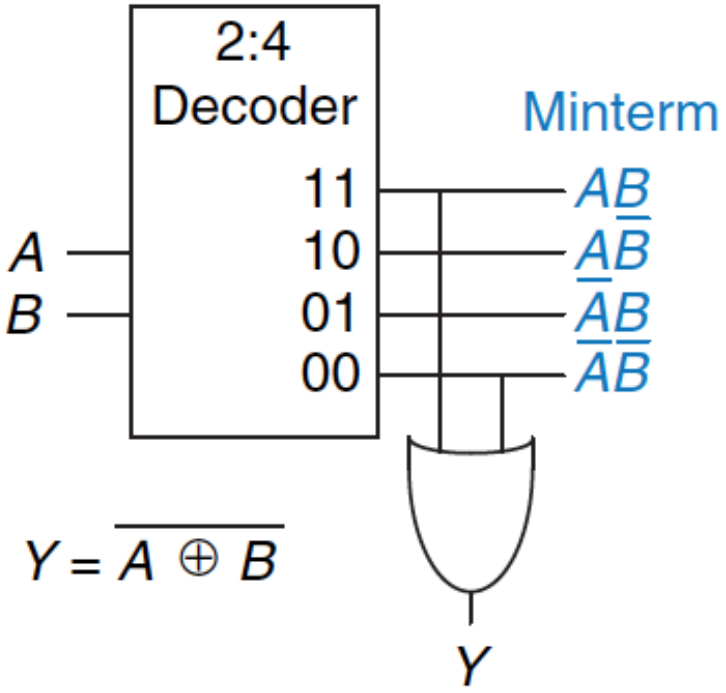


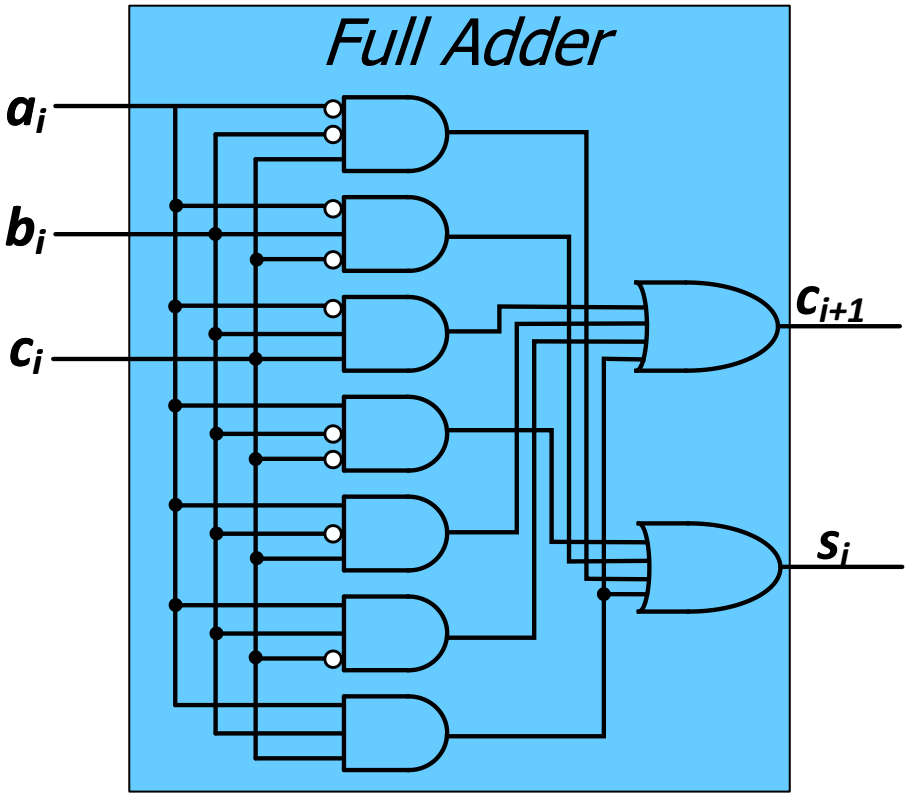
Figure 2.65 Logic function using decoder

Read H&H Chapter 2.8

Logic Simplification using Boolean Algebra Rules

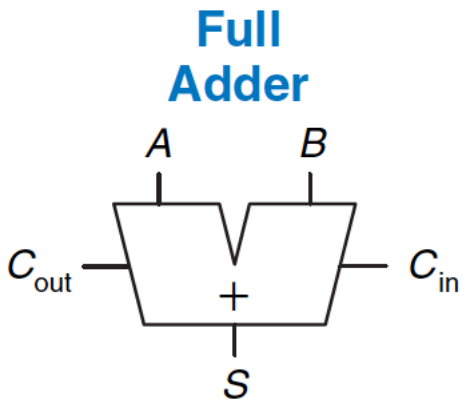
© Osmonov M.S.
IUCA

Recall: Full Adder in SOP Form Logic



a_i	b_i	$carry_i$	$carry_{i+1}$	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Goal: Simplified Full Adder



$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + AC_{in} + BC_{in}$$

C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

How do we simplify Boolean logic?

Quick Recap on Logic Simplification

- The original Boolean expression (i.e., logic circuit) may not be optimal

$$F = \sim A(A + B) + (B + AA)(A + \sim B)$$

- Can we reduce a given Boolean expression to an equivalent expression with fewer terms?

$$F = A + B$$

- The goal of logic simplification:
  - Reduce the number of gates/inputs
  - Reduce implementation cost

A basis for what the automated design tools are doing today

Logic Simplification

- Systematic techniques for simplifications
  - amenable to automation

Key Tool: The Uniting Theorem — $F = A\overline{B} + AB$

A	B	F
0	0	0
0	1	0
1	0	1
1	1	1

$$F = A\overline{B} + AB = A(\overline{B} + B) = A(1) = A$$

B's value changes within the rows where F==1 ("ON set")

A's value does NOT change within the ON-set rows

If an input (B) can change without changing the output, that input value is not needed

→ B is eliminated, A remains

A	B	G
0	0	1
0	1	0
1	0	1
1	1	0

$$G = \overline{A}\overline{B} + A\overline{B} = (\overline{A} + A)\overline{B} = \overline{B}$$

B's value stays the same within the ON-set rows

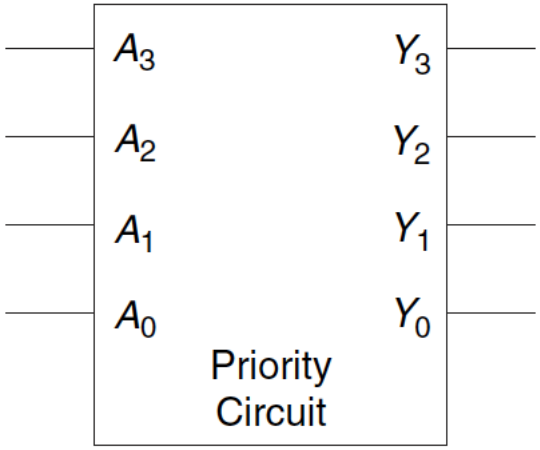
A's value changes within the ON-set rows

→ A is eliminated, B remains

Essence of Simplification:
Find two element subsets of the ON-set where only one variable changes its value. This single varying variable *can be eliminated!*

Logic Simplification Example: Priority Circuit

- Priority Circuit
  - Inputs: “Requestors” with priority levels
  - Outputs: “Grant” signal for each requestor
  - Example 4-bit priority circuit



A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	0	0

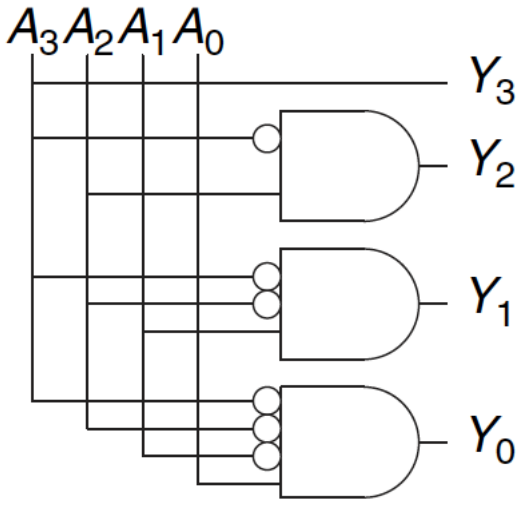
Simplified Priority Circuit

- Priority Circuit
  - Inputs: “Requestors” with priority levels
  - Outputs: “Grant” signal for each requestor
  - Example 4-bit priority circuit

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	0
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	0
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	1	0	0	0
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	1	0	0	0

A_3	A_2	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	X	0	0	1	0
0	1	X	X	0	1	0	0
1	X	X	X	1	0	0	0

Figure 2.29 **Priority circuit truth table with don’t cares (X’s)**



X (Don’t Care) means *I don’t care what the value of this input is*

Logic Simplification: Karnaugh Maps (K-Maps)

© Osmonov M.S.
IUCA

Karnaugh Maps are Fun…

- A pictorial way of minimizing circuits by visualizing opportunities for simplification
- They are for you to **study on your own**…

- See Backup Slides
- Read H&H Section 2.7
- Watch videos of Lectures 5 and 6 from 2019 DDCA course:
  - https://youtu.be/0ks0PeaOUjE?list=PL5Q2soXY2Zi8J58xLKBNFQFHRO3GrXxA9&t=4570
  - https://youtu.be/ozs18ARNG6s?list=PL5Q2soXY2Zi8J58xLKBNFQFHRO3GrXxA9&t=220

Backup Slides on Karnaugh Maps (K-Maps)

© Osmonov M.S.
IUCA

Complex Cases

- One example

$$Cout = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$$

- Problem
  - Easy to see how to apply Uniting Theorem…
  - Hard to know if you applied it in all the right places…
  - …especially in a function of many more variables

- Question
  - Is there an easier way to find potential simplifications?
  - i.e., potential applications of Uniting Theorem…?

- Answer
  - Need an intrinsically *geometric* representation for Boolean f( )
  - Something we can draw, see…

K-map Rules

- **What can be legally combined (circled) in the K-map?**
  - Rectangular groups of size 2^k for any integer k
  - Each cell has the same value (1, for now)
  - All values must be adjacent
    - Wrap-around edge is okay
- **How does a group become a term in an expression?**
  - Determine which literals are constant, and which vary across group
  - Eliminate varying literals, then AND the constant literals
    - constant 1 → use X, constant 0 → use $\overline{X}$
- **What is a good solution?**
  - Biggest groupings → eliminate more variables (literals) in each term
  - Fewest groupings → fewer terms (gates) all together
  - OR together all AND terms you create from individual groups

Karnaugh Map

- Karnaugh Map (K-map) method
  - K-map is an alternative method of representing the truth table that helps visualize adjacencies in up to 6 dimensions
  - Physical adjacency ↔ Logical adjacency

2-variable K-map

A \ B	0	1
0	00	01
1	10	11

3-variable K-map

A \ BC	00	01	11	10
0	000	001	011	010
1	100	101	111	110

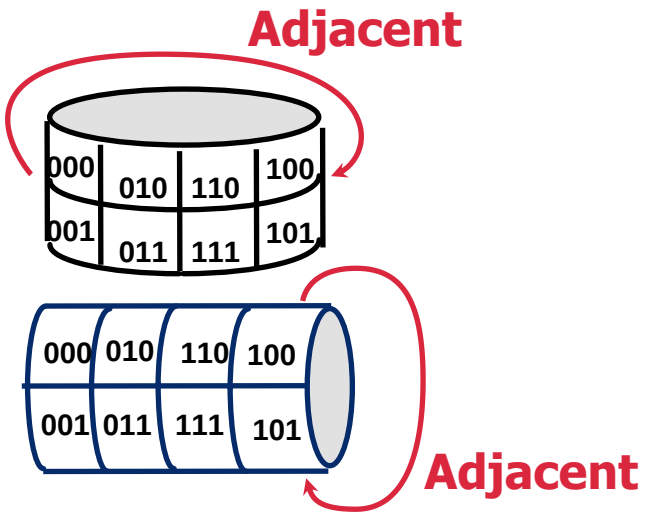
4-variable K-map

AB \ CD	00	01	11	10
00	0000	0001	0011	0010
01	0100	0101	0111	0110
11	1100	1101	1111	1110
10	1000	1001	1011	1010

Numbering Scheme: **00, 01, 11, 10 is called a "Gray Code" — only a *single bit (variable) changes* from one code word and the next code word**

Karnaugh Map Methods

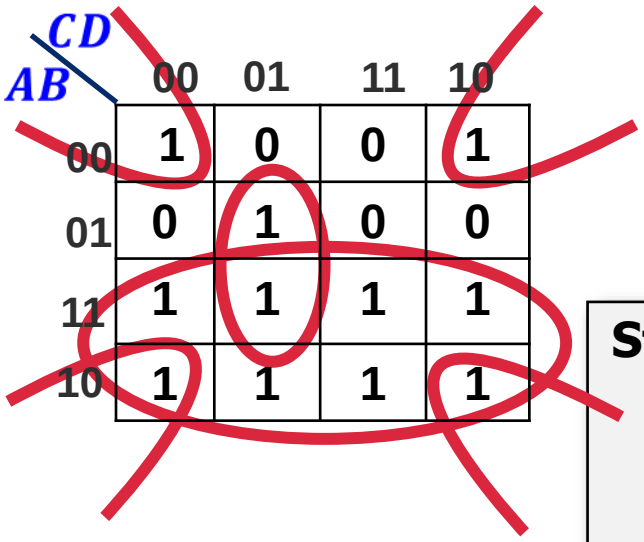
A \ BC	00	01	11	10
0	000	001	011	010
1	100	101	111	110



0

K-map adjacencies go "around the edges"
Wrap around from first to last column
Wrap around from top row to bottom row

K-map Cover - 4 Input Variables



$$F(A,B,C,D) = \sum m(0,2,5,8,9,10,11,12,13,14,15)$$

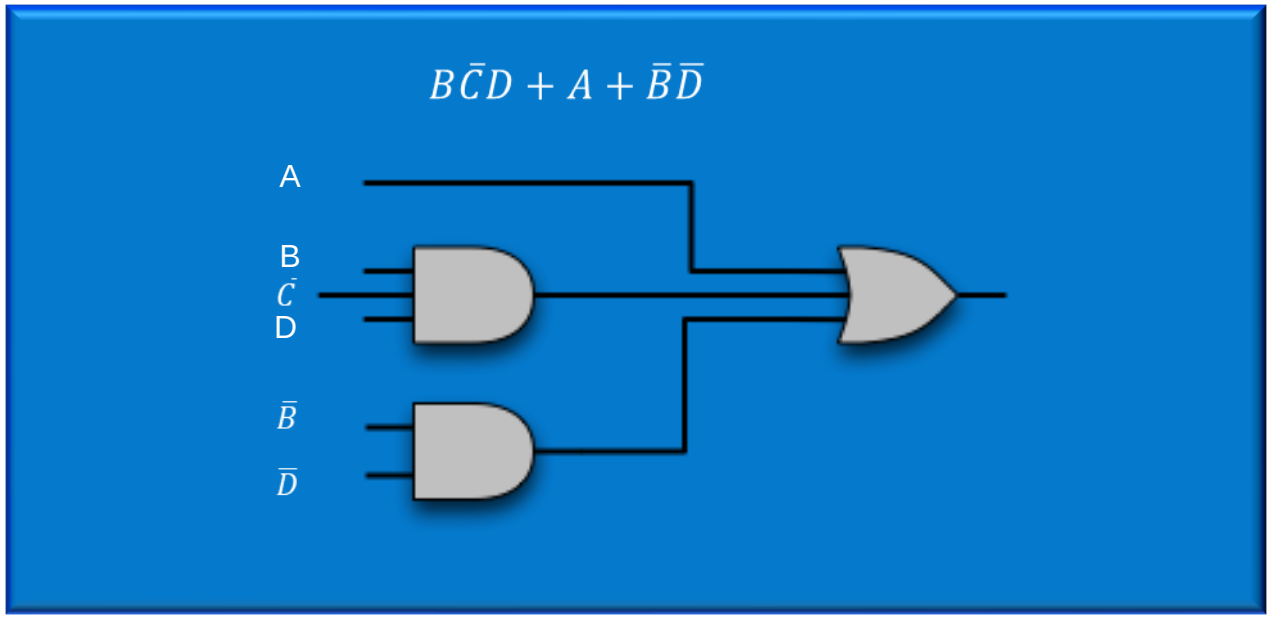
$$F = A + \bar{B}\bar{D} + B\bar{C}D$$

Strategy for "circling" rectangles on Kmap:

Biggest "oops!" that people forget:

Example: As Logic Gates

- **What does this solution look like as gates?**



Logic Minimization Using K-Maps

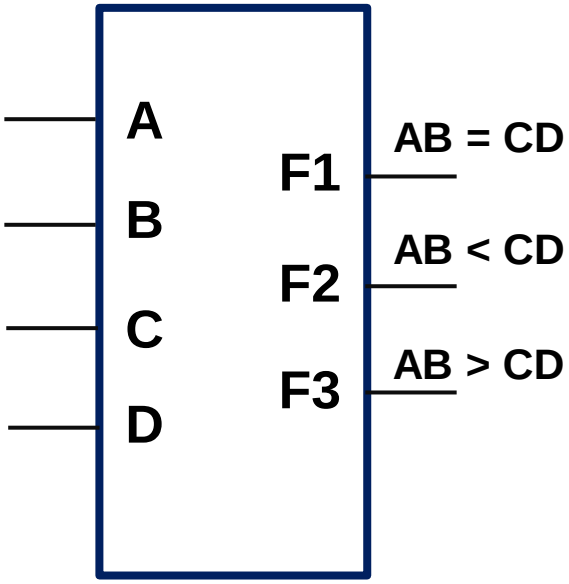
- Very simple guideline:
  - Circle all the rectangular blocks of 1's in the map, using the fewest possible number of circles
    - Each circle should be as large as possible
  - Read off the implicants that were circled

- More formally:
  - A Boolean equation is minimized when it is written as a sum of the fewest number of prime implicants
  - Each circle on the K-map represents an implicant
  - The largest possible circles are prime implicants

K-map Rules

- **What can be legally combined (circled) in the K-map?**
  - Rectangular groups of size 2^k for any integer k
  - Each cell has the same value (1, for now)
  - All values must be adjacent
    - Wrap-around edge is okay
- **How does a group become a term in an expression?**
  - Determine which literals are constant, and which vary across group
  - Eliminate varying literals, then AND the constant literals
    - constant 1 → use X, constant 0 → use $\overline{X}$
- **What is a good solution?**
  - Biggest groupings → eliminate more variables (literals) in each term
  - Fewest groupings → fewer terms (gates) all together
  - OR together all AND terms you create from individual groups

K-map Example: Two-bit Comparator



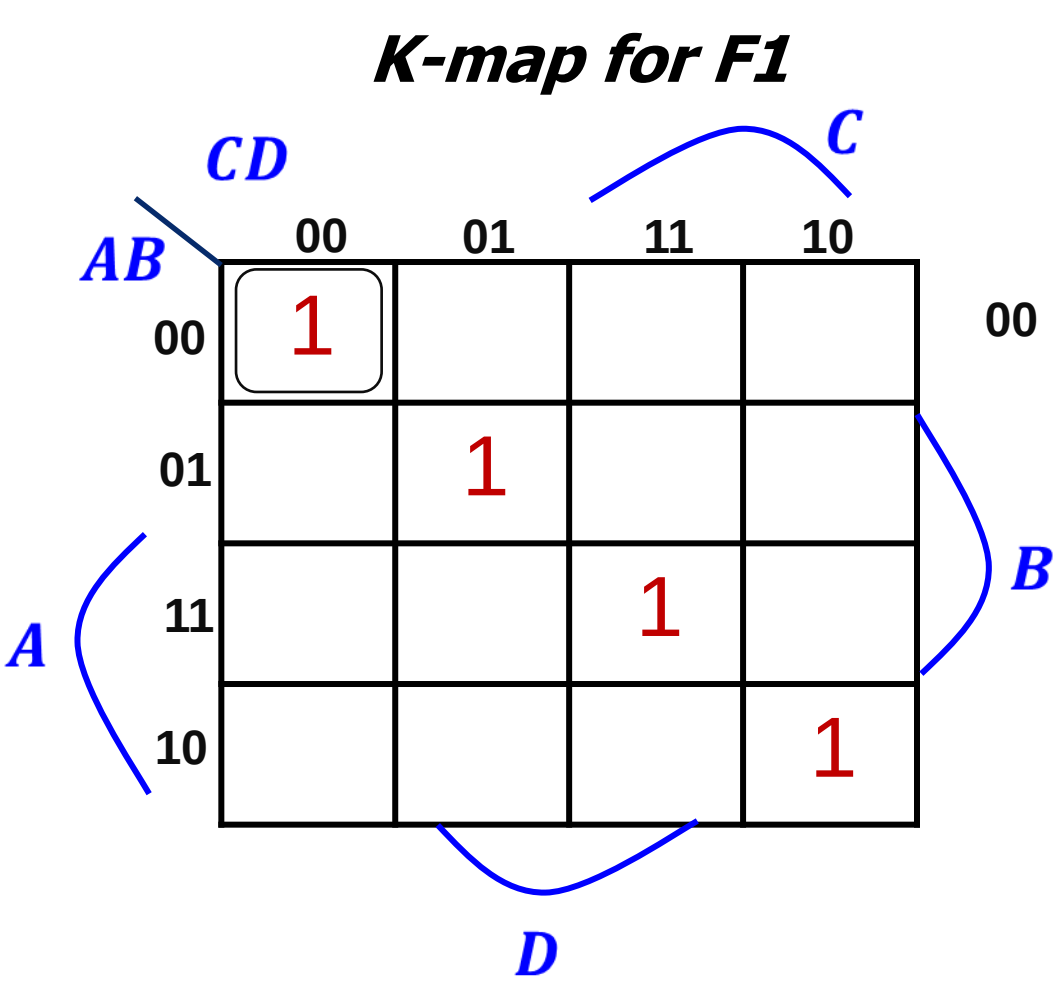
Design Approach:

Write a 4-Variable K-map for each of the 3output functions

A	B	C	D	F1	F2	F3
0	0	0	0	1	0	0
0	0	0	1	0	1	0
0	0	1	0	0	1	0
0	0	1	1	0	1	0
0	1	0	0	0	0	1
0	1	0	1	1	0	0
0	1	1	0	0	1	0
0	1	1	1	0	1	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	1	0	0
1	0	1	1	0	1	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	1	0	0

© Osmonov M.S.
KRSU

K-map Example: Two-bit Comparator (2)

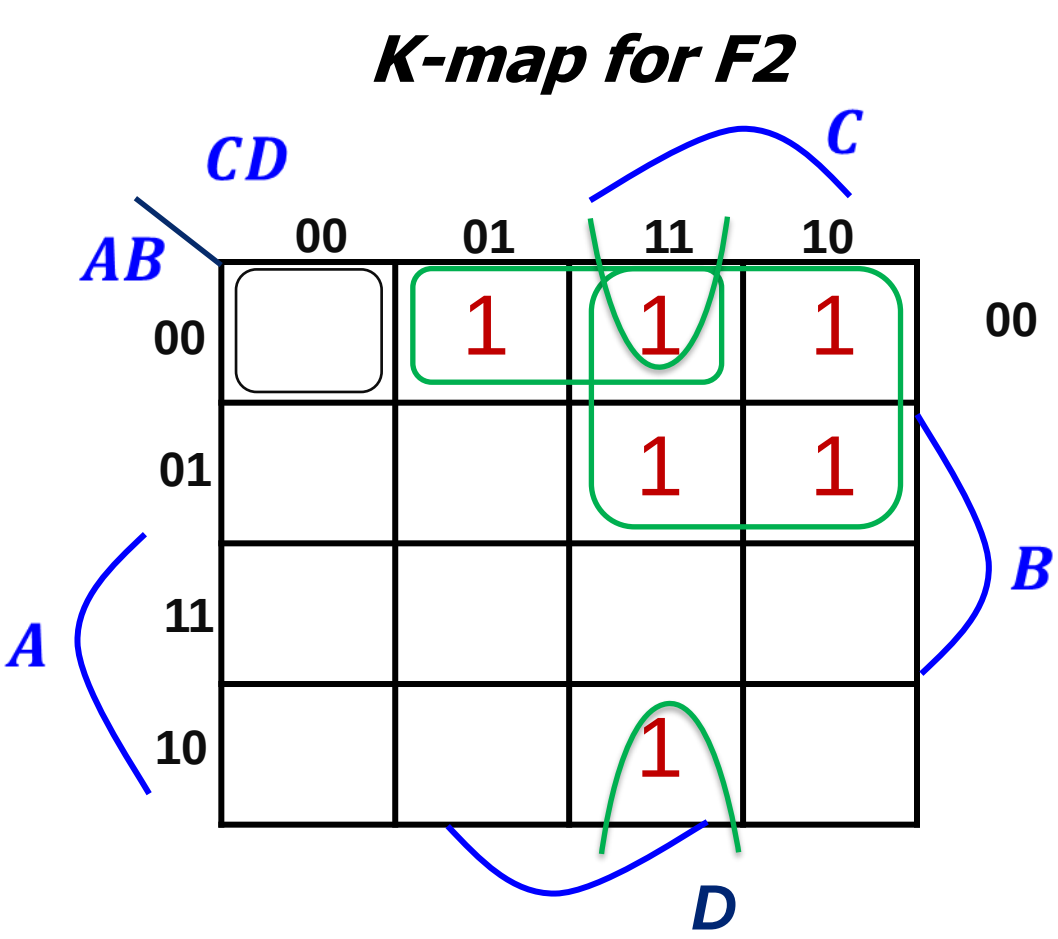


A	B	C	D	F1	F2	F3
0	0	0	0	1	0	0
0	0	0	1	0	1	0
0	0	1	0	0	1	0
0	0	1	1	0	1	0
0	1	0	0	0	0	1
0	1	0	1	1	0	0
0	1	1	0	0	1	0
0	1	1	1	0	1	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	1	0	0
1	0	1	1	0	1	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	1	0	0

F1 = A'B'C'D' + A'BC'D + ABCD + AB'CD'

© Osmonov M.S.
KRSU

K-map Example: Two-bit Comparator (3)



A	B	C	D	F1	F2	F3
0	0	0	0	1	0	0
0	0	0	1	0	1	0
0	0	1	0	0	1	0
0	0	1	1	0	1	0
0	1	0	0	0	0	1
0	1	0	1	1	0	0
0	1	1	0	0	1	0
0	1	1	1	0	1	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	1	0	0
1	0	1	1	0	1	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	1	0	0

F2 = A'C + A'B'D + B'CD

F3 = ? (Exercise for you)

K-maps with “Don’t Care”

- Don’t Care really means *I don’t care what my circuit outputs if this appears as input*
  - You have an engineering choice to use DON’T CARE patterns intelligently as 1 or 0 to better simplify the circuit

A	B	C	D	F1	G
•	•	•			
0	1	1	0	X	X
0	1	1	1		
1	0	0	0	X	X
1	0	0	1		
•	•	•			

I can pick 00, 01, 10, 11 independently of below

I can pick 00, 01, 10, 11 independently of above

Example: BCD Increment Function

- BCD (Binary Coded Decimal) digits
  - Encode decimal digits 0 - 9 with bit patterns 0000_2 — 1001_2
  - When incremented, the decimal sequence is 0, 1, …, 8, 9, 0, 1

A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	0	0	0	0
1	0	1	0	x	x	x	x
1	0	1	1	x	x	x	x
1	1	0	0	x	x	x	x
1	1	0	1	x	x	x	x
1	1	1	0	x	x	x	x
1	1	1	1	x	x	x	x

These input patterns should never be encountered in practice (hey -- it's a BCD number!)
So, associated output values are "Don't Cares"

K-map for BCD Increment Function

$$\begin{array}{r} ABCD \\ +\quad\ 1 \\ \hline WXYZ \end{array}$$

W

AB \ CD	00	01	11	10
00				
01			1	
11	X	X	X	X
10	1		X	X

X

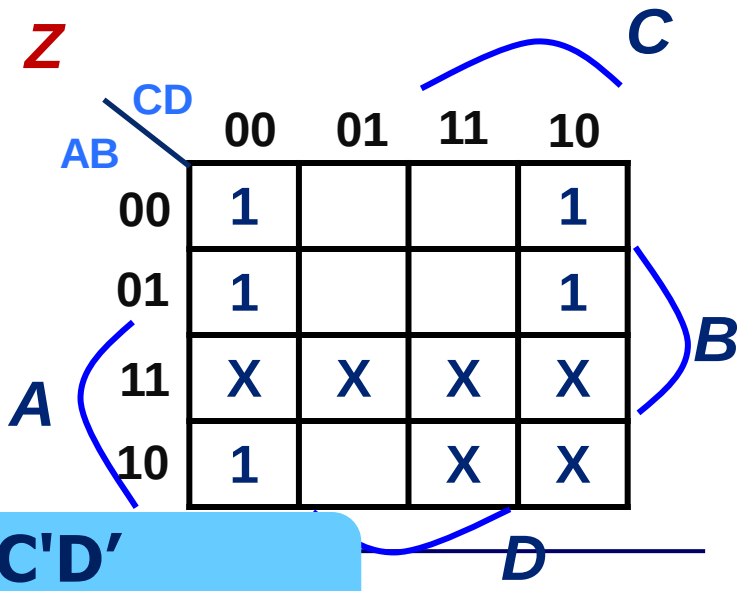
AB \ CD	00	01	11	10
00			1	
01	1	1		1
11	X	X	X	X
10			X	X

Y

AB \ CD	00	01	11	10
00		1		1
01		1		1
11	X	X	X	X
10			X	X

Z

AB \ CD	00	01	11	10
00	1			1
01	1			1
11	X	X	X	X
10	1		X	X



Z (without don't cares) = **A'D' + B'C'D'**

Z (with don't cares) = **D'**

K-map Summary

- Karnaugh maps as a formal systematic approach for logic simplification
- 2-, 3-, 4-variable K-maps
- K-maps with “Don’t Care” outputs
- H&H Section 2.7

The lecture is over



Thank you for your attention!!!